

计算机系统综合实践 指导书



温州大学计算机与人工智能学院

任课教师：徐玉

目录

1. 智能路由器概述.....	1
1.1 智能路由器简介.....	1
1.2 OpenWrt 简介.....	1
2. 智能路由器硬件.....	2
2.1 MIPS 架构处理器.....	2
2.2 JS7628 智能路由器开发板.....	2
3. 智能路由器开发环境搭建.....	6
3.1 准备 Linux 操作系统.....	6
3.2 搭建 OpenWrt 开发环境.....	6
3.2.1 配置编译环境.....	6
3.2.2 下载 OpenWrt 源代码.....	6
3.2.3 修改硬件配置.....	8
3.2.4 OpenWrt 系统功能配置.....	9
3.2.5 OpenWrt 源码编译.....	11
3.2.6 安装 OpenWrt SDK.....	12
3.3 刷新 OpenWrt 固件.....	13
3.3.1 用 OpenWrt 升级页面烧写.....	13
3.3.2 用 uboot 升级页面烧写.....	14
3.3.3 用 uboot+串口+网络烧写.....	15
3.4 调试串口连接.....	15
3.5 SSH 远程登录.....	16
3.6 开发板与计算机的文件传输.....	17
3.6.1 用 SCP 协议传输文件.....	17
3.6.2 用 NFS 服务传输文件.....	17
3.6.3 用 FTP 传输文件.....	17
3.7 连接外网.....	17
4. OpenWrt 基础.....	18
4.1 软件包安装.....	18
4.1.1 通过 OPKG 在线安装.....	18
4.1.2 通过 OpenWrt 配置页面安装.....	20
4.1.3 离线安装.....	20

4.2	UCI 配置命令	21
4.2.1	UCI 简介	21
4.2.2	UCI 配置文件语法	21
4.2.3	统一配置原理	22
4.2.4	UCI 命令	22
4.2.5	UCI API 接口	23
4.3	软件包开发	24
4.3.1	软件包工程建立	24
4.3.2	读取软件包配置	27
4.3.3	通过 LuCI 修改软件包配置	30
4.3.4	串口消息的定时输出	33
4.3.5	软件包的自动运行	37
5.	智能路由器应用实例	40
5.1	智能路由器媒体中心	40
5.2	智能物联网设备	42
5.2.1	传感器模拟器	42
5.2.2	智能路由器与物联网设备模拟器的通讯	43
5.2.3	腾讯云物联网设备端 C SDK 移植	44
5.2.4	云端产品建立	45
5.2.5	连接 MQTT 云端服务并构造数据模板客户端	47
5.2.6	注册设备数据模板属性	48
5.2.7	获取云端数据模板属性	50
5.2.8	向云端上报数据	50
5.2.9	响应云端下行的数据模板属性控制消息	51

1. 智能路由器概述

1.1 智能路由器简介

智能路由器也就是智能化管理的路由器，通常具有独立的操作系统，可以由用户自行安装各种应用，自行控制带宽、在线人数、浏览网页和在线时间，同时拥有强大的 USB 共享功能，真正做到网络 and 设备的智能化管理。当前主流的智能路由器操作系统有 OpenWrt、Tomato WRT、DD-WRT 等。

1.2 OpenWrt 简介

OpenWrt 专为嵌入式设备所研发，是一个高度模块化、高度自动化的嵌入式 Linux 系统，拥有强大的网络组件和扩展性，是现在很多智能路由设备的操作系统。OpenWrt 不同于 Tomato WRT 和 DD-WRT，它是一个从零开始编写的、功能齐全的、容易修改路由器操作系统，采用了一个非常灵活并且开放式的架构，因此吸引了全世界大量的厂商和开发者，同时也支持数量繁多的芯片，包括 x86、ARM、MIPS 等。开发者使用 OpenWrt，只需要直接通过简单的编译，就可以将一套软件编译出不同芯片的版本，烧写到自己的路由器上就拥有了一台 Linux 服务器。OpenWrt 的成功之处还在于它的文件系统是可写的，开发者无需在每次修改后重新编译系统，可以像 PC 机上的 Linux 系统一样，通过 OPKG 增强型安装技术，一个命令就可以将已预编译好的软件安装到系统中，这些都令他更像一个小型的 Linux 计算机系统。

OpenWrt 的主要功能包括网络功能、系统管理功能和状态监控功能。网络管理功能是路由器的核心功能，主要实现网络接口设置与管理、DHCP 协议、DNS、静态与组播路由、网络诊断、防火墙、IP 带宽控制、设备即插即用等功能。系统管理功能是路由器除网络管理之外的其他管理功能，主要实现主机名称设置、日志服务器设置、NTP、密码设置、远程安全登录（SSH）、软件管理、配置备份、进程启动与定时任务管理、系统属性设置等功能。状态监控主要用于监控路由器的当前状态，包括固件版本、运行时间、平均负载、内存占用、网络状态、DHCP 用户、无线用户、防火墙状态、路由转发表、ARP 表、系统日志、内核启动日志、CPU 使用率、内存使用率等。

2. 智能路由器硬件

2.1 MIPS 架构处理器

近 10 年来，路由器所采用的 CPU 主要都是基于 MIPS 架构设计的。MIPS 采用 RISC 精简指令集设计技术，相比 CISC 复杂指令集技术具有更高的执行效率，我国的龙芯便是 MIPS 架构下的一种处理器。

目前路由器芯片普遍采用 SoC（System on Chip，片上系统）设计，很多路由器芯片产商喜欢把它叫做 RoC，这些厂家将内存控制、外围 I/O 接口、交换机芯片、Wi-Fi 芯片、加密芯片、SATA 接口、PCIE 接口、USB 接口等所有硬件统统集成到一个手指甲大小的芯片当中，而其功耗只有 1~3W，频率达到了 300-800MHz。目前常见的路由器芯片产商主要有 Atheros、Broadcom、Ralink。Atheros 是全球路由芯片的顶级公司，其创始人是 MIPS 架构的发明者约翰·轩尼斯，当前该公司的芯片主要用于高端及企业级产品线，目前已被高通收购。Broadcom 的无线路由芯片产品以稳定可靠著称，其路由芯片也大多是 MIPS 架构的。Ralink 也是一家常年耕耘在路由器领域的芯片产商，其芯片价格相对经济，国内上市的家庭智能路由器大多采用了该公司方案，目前已被联发科收购。

2.2 JS7628 智能路由器开发板

JS7628 是杭州卓钛科技有限公司开发的基于联发科 MT7628NN/AN SOC 芯片方案的智能路由器开发板。MT7628 是联发科 2014 年底发布的业内新一代 2T2R 802.11n Wi-Fi AP 路由器系统单芯片解决方案（SOC）。该芯片为智能家庭内的数据、语音和影像应用提供高数据传输率，且耗电量远低于市场上的同类产品。JS7628 的特性如下：

- 580Mhz 系统主频、256MB DDR2 RAM、32MB SPI NOR Flash
- 无线协议 802.11 b/g/n，2T2R，最高 300 Mbps 无线传输速率
- 3 个 100/10Mbps 支持自动翻转的 LAN/WAN 以太网口
- 4 路标准的高速 USB2.0 Host 接口、1 路 TTL/RS232 串口、1 路 TTL/485 串口、1 路 USB Mini 调试串口
- 5 个可配置 LED 普通网络/系统指示灯，4 路 PWM LED 灯
- 1 个 Micro SD/TF 卡接口

- 1 路音频输出, 1 路音频输入接口
- 1 路红外接收, 1 路红外发射
- 15 路功能复用的 GPIO 接口

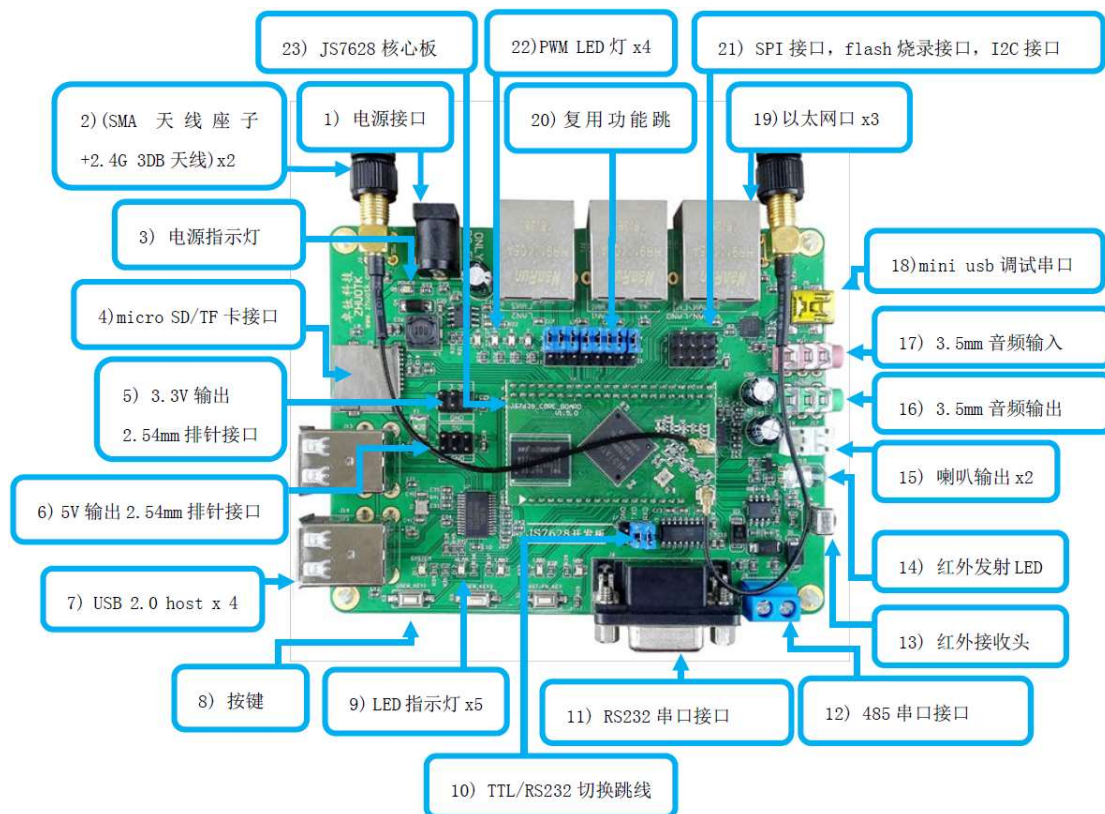


图 2-1 JS7628 的硬件资源

图 2-1 标识出了 JS7628 的硬件资源，其中：

- 1) 电源接口。**请务必采用开发板自带的 DC 5V 2A 电源，用错电源容易导致开发板损坏。**
- 2) SMA 天线座子 x2, 2.4G 3DBi 天线 x2。双天线具有 2T2R（双发双收）300Mbps wifi 的高速效果，用单天线也能正常工作，不过 wifi 速率会受影响，只能实现 1T1R（单发单收）的功能。
- 3) 电源指示灯。接通电源后该灯亮。
- 4) Micro SD/TF 卡接口。用于扩展存储空间，存储图片、音视频、安装超大程序。
- 5) 3.3V 输出，2.54mm 排针接口。方便接其他 3.3V DC 供电的模块，双排针上排为正下排为地，**切勿短接或接错。**
- 6) 5V 输出，2.54mm 排针接口。方便接其他 5V DC 供电的模块，双排针上排为正下排为地，**切勿短接或接错。**
- 7) USB 2.0 Host x 4。4 个标准 USB 2.0 Host 接口，可以接多个 USB 外设，如 U 盘、USB

摄像头、USB 声卡、USB 4G 网卡、USB 鼠标、USB 键盘等。

8) 按键。从左到右依次是：

- “RST/FN_KEY”：复位/恢复出厂设置按键。开发板启动完成后，短按（范围小于 2s），开发板重启。长按（范围 5s~30s，一般 10s 左右），开发板恢复到出厂设置，建议第 1 次拿到开发板后执行此操作。
- “USER_KEY2”：功能未定义，用户也可以参考《JS76x8 开发板 openwrt 入门教程》中“添加修改按键功能”一节，自行配置按键功能。
- “USER_KEY1”：同上述。

9) LED 指示灯，从左到右依次是：

- “system”：系统运行指示灯。
- “wlan”：wifi 通讯指示灯。
- “port2”：网口 2（PORT2）通讯指示灯，该网口有连接灯亮起，有通讯灯闪烁。
- “port1”：网口 1（PORT1）通讯指示灯，该网口有连接灯亮起，有通讯灯闪烁。
- “port0”：网口 0（PORT0）通讯指示灯，该网口有连接灯亮起，有通讯灯闪烁。

10) TTL/RS232 切换跳线，插上跳线帽，则启用 RS232 接口。拔掉跳线帽，则可接 3.3V TTL 串口。默认跳线帽已插上。

11) RS232 串口接口，需按照 10) 的说明启用该接口。串口 1，系统中的设备号是“/dev/ttyS1”。

12) 485 串口接口，需按照 21) 的说明启用该接口。串口 2，系统中的设备号是“/dev/ttyS2”。

13) 红外接收头，开发板的提供相应的驱动和演示程序源码。

14) 红外发射 LED，开发板的提供相应的驱动和演示程序源码。

15) 喇叭输出，2 个 1W 喇叭左右声道输出，可以配合配套喇叭，实现声音外放功能。

16) 3.5mm 音频输出，可以接普通耳机，音箱输入，支持 24bit 高品质 HIFI 音乐播放，也可以实现通话功能。

17) 3.5mm 音频输入，可以接普通麦克风，支持录音、通话功能等功能。

18) mini usb 调试串口，开发板通过板载的 USB 转串口芯片（CP2104），实现了核心板的调试串口转到 USB 接口功能。只需外接一根 mini USB 线，即可对开发板进行串口调试、下载程序。

19) 以太网口，从左到右依次是以太网口 PORT2、PORT1、PORT0，这 3 个网口默认是 LAN 口功能，可以接任意一个，登录开发板的配置页面“192.168.1.251”。这 3 个网口也可配

置成为 WAN 口，实现普通路由器的 WAN 口功能，也可以实现多个 WAN 口多拨。

20) 复用功能跳线，可以实现网口和 PWM、GPIO、UART2（TTL/485 串口）硬件连接切换，并且需要配合相应的固件才能实现软件功能切换，请烧录资料中提供的相应固件。

21) SPI 接口，flash 烧录接口，I²C 接口。

22) PWM LED 灯，4 个可由 PWM 信号控制的 LED 灯。

23) JS7628 核心板。

3. 智能路由器开发环境搭建

3.1 准备 Linux 操作系统

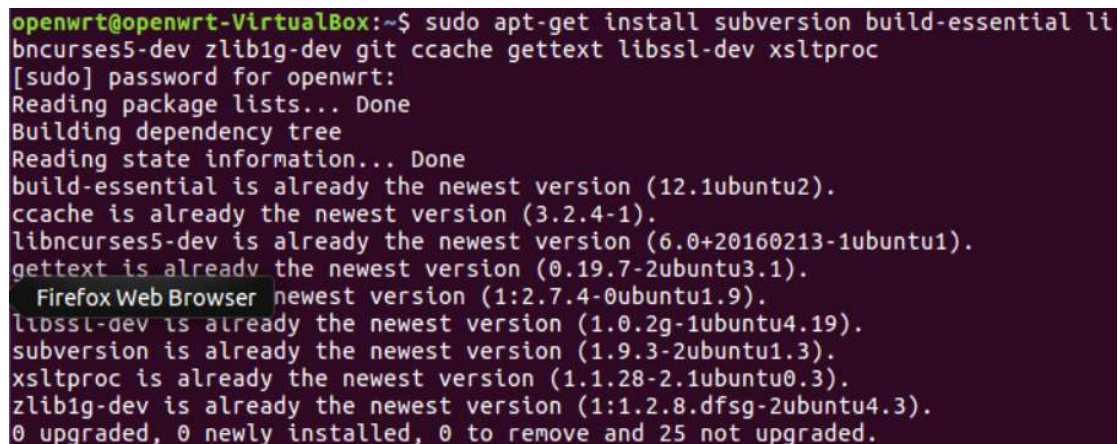
Linux 操作系统采用 64 位 Ubuntu16.04.07，可以到 <ftp://10.5.12.106/> 计算机系统综合实践/资源下载/目录下载 ubuntu-16.04.7-desktop-amd64.iso，自己在 VMWare 或 VirtualBox 虚拟机下进行安装。同时也可以在该目录下载已经安装好的 VirtualBox 虚拟机文件 ubuntu-openwrt-student.ova 直接导入使用，用户名和密码均为 openwrt。

3.2 搭建 OpenWrt 开发环境

3.2.1 配置编译环境

为了能正常地编译 OpenWrt 源码，我们需要在 Ubuntu 系统中安装一些软件。打开一个终端，在 ubuntu 终端下面输入如下命令：

```
sudo apt-get install subversion build-essential libncurses5-dev zlib1g-dev gawk git ccache  
gettext libssl-dev xsltproc
```



```
openwrt@openwrt-VirtualBox:~$ sudo apt-get install subversion build-essential libncurses5-dev zlib1g-dev git ccache gettext libssl-dev xsltproc  
[sudo] password for openwrt:  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
build-essential is already the newest version (12.1ubuntu2).  
ccache is already the newest version (3.2.4-1).  
libncurses5-dev is already the newest version (6.0+20160213-1ubuntu1).  
gettext is already the newest version (0.19.7-2ubuntu3.1).  
Firefox Web Browser newest version (1:2.7.4-0ubuntu1.9).  
libssl-dev is already the newest version (1.0.2g-1ubuntu4.19).  
subversion is already the newest version (1.9.3-2ubuntu1.3).  
xsltproc is already the newest version (1.1.28-2.1ubuntu0.3).  
zlib1g-dev is already the newest version (1:1.2.8.dfsg-2ubuntu4.3).  
0 upgraded, 0 newly installed, 0 to remove and 25 not upgraded.
```

系统将提示输入系统用户密码。输入正确密码后，系统将开始安装指定文件。**注意：如果未完成安装以上的软件，可能会造成编译 openwrt 源码时出错，请务必安装完全。**

3.2.2 下载 OpenWrt 源代码

OpenWrt 源代码可以通过两种方式获得，一种是从 OpenWrt 官网下载源代码，然后根

据开发板的硬件设计对源代码进行修改。另外一种直接使用开发板厂商提供的 OpenWrt 源代码（也是基于官方源代码修改而来，ftp 资源下载目录下的 openwrt_cc_mt76x8_jotale_source_20201130.tar.bz2 文件）。**建议采用第 2 种方式获取源代码，如果要采用第一种方式，请参照《JS76x8 开发板 OpenWrt 入门教程 v1.4.14》文件 4.3.2.1 节。**将该安装包拷贝到当前登录 ubuntu 用户的工作目录中（即~/目录下），并执行如下命令：

- `tar xjvf openwrt_cc_mt76x8_jotale_source_20201130.tar.bz2` //解压文件
- `cd openwrt_cc_mt76x8_jotale_source_20201130` //进入源代码目录
- 修改 feeds.conf.default 文件，将所有源替换为：

```
src-git packages https://github.com.cnpmjs.org/openwrt/packages.git;for-15.05
src-git luci https://github.com.cnpmjs.org/openwrt/luci.git;for-15.05
src-git routing https://github.com.cnpmjs.org/openwrt-routing/packages.git;for-15.05
src-git telephony https://github.com.cnpmjs.org/openwrt/telephony.git;for-15.05
src-git management https://github.com.cnpmjs.org/openwrt-
management/packages.git;for-15.05
src-git extra git://github.com/nanpuyue/openwrt-extra.git
```

- `./scripts/feeds update -a` //更新依赖包
- `./scripts/feeds install -a` //安装依赖的软件包，**必须执行，否则会出错**

如果因网络问题上述步骤没有成功，请将“github.com.cnpmjs.org”改成“github.com”后再试。完成后，OpenWrt 的源文件如下所示：

```
BSDmakefile  dl      feeds.conf.default  Makefile  rules.mk  target  tools
config       docs    include            package  scripts  tmp      toolchain
Config.in    feeds  LICENSE            README   staging_dir
```

其中：

- **scripts** 目录存放了一些脚本，使用了 bash、Python、perl 等多种脚本语言。编译过程中,用于第三方软件包管理的 feeds 文件也是在这个目录当中。在编译过程中，使用到的脚本也统一放在这个目录中。
- **tools** 目录里包含了获取和编译这些工具的命令。软件包里面有 Makefile 文件，有的还包含了 patch。每个 Makefile 当中都有一句`$(eval $(call HostBuild))`，这表明编译这个工具是为了在主机上使用的。
- **config** 目录存放着整个系统的配置文件。
- **docs** 目录包含了整个宿主机的文件源码的介绍，里面还有 Makefile 为目标系统生成 docs。使用 `make -C docs/` 可以为目标系统生成文档。

- **toolchain** 目录存放编译交叉编译链的软件包。包括:binutils、gcc、libc 等等。
- **target** 目录中定义了各个 CPU 平台 firmware 和 kernel 的编译过程。
- **package** 里存放 openwrt 系统中适用的软件包, 包含针对各个软件包的 Makefile。
openwrt 定义了一套 Makefile 模板, 各软件参照这个模板定义了自己的信息, 如软件包的版本、下载地址、编译方式、安装地址等。在二次开发过程中, 这个文件夹我们会经常打交道。事实上,通过./scripts/feed update -a 和./scripts/feed install -a 的软件包也会存放在这个目录之中。
- **include** 里存放 openwrt 的很多 Makefile, 文件名为 *.mk, 这些文件定义了编译过程。这里的文件上是在 Makefile 里被 include 的, 类似于库文件。
- **feeds** 是 OpenWrt 附加软件包管理器的扩展包索引目录, 默认的 feeds 下载有 packages、management、luci、routing、telephony。如要下载其他的软件包, 需要打开源码根目录下面的 feeds.conf.default 文件, 去掉相应软件包前面的 # 号, 然后更新源 (./scripts/feeds update -a) 并安装包 (./scripts/feeds install -a)。
- **Makefile**, 在顶层目录执行 make 命令的入口文件。
- **rules.mk**, 定义了 Makefile 中使用的一些通用变量和函数。
- **Config.in**, 在 include/toplevel.mk 中我们可以看到,这是和 make menuconfig 相关联的文件。
- **feeds.conf.default**, 在这个文件中可以配置下载第三方一些软件包时所使用的地址。
- **LICENSE & README**, 即软件许可证和软件基本说明。

3.2.3 修改硬件配置

开发板的硬件配置相比芯片产商提供的标准配置不可避免会存在差异, 因此需要根据开发板的硬件对 openwrt 源码/target/linux/ramips/dts/MT7628.dts 文件进行修改, 具体修改内容如下:

- 修改支持的内存大小为 256MB, 在 dts 文件第 16-18 行, 将相应行注释去掉即可启用相应内存配置。**注意: 选择和硬件不匹配的配置, 可能会导致系统运行不正常。**

```
14     memory@0 {
15         device_type = "memory";
16         reg = <0x0 0x10000000>; //256MB RAM
17         // reg = <0x0 0x8000000>; //128MB RAM
18         // reg = <0x0 0x4000000>; //64MB RAM
19     };
```

- 修改支持的 Flash 大小，在 dts 文件第 123-125 行，将相应行注释去掉即可启用相应的 Flash 配置。**注意：选择和硬件不匹配的配置，可能会导致系统运行不正常。**

```

120
121         partition@50000 {
122             label = "firmware";
123             // reg = <0x50000 0x7b0000>; //8MB flash
124             // reg = <0x50000 0xfb0000>; //16MB flash
125             reg = <0x50000 0x1fb0000>; //32MB flash
126         };

```

- 修改 MT7268 芯片 SPIS 管脚默认配置为“pwm_uart2”功能，在 dts 文件第 47-50 行。

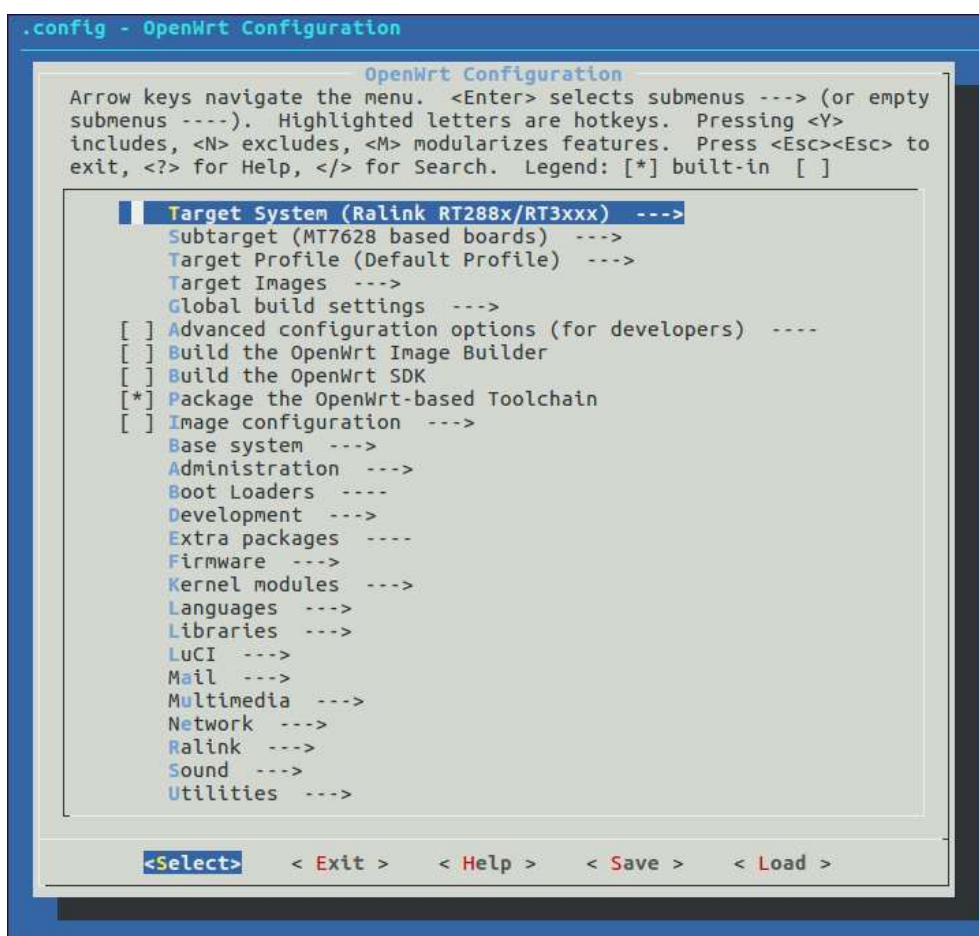
```

47         spis {
48             ralink,group = "spis";
49             ralink,function = "pwm_uart2";
50         };

```

3.2.4 OpenWrt 系统功能配置

OpenWrt 也具有像经典的 Linux 的 make menuconfig 功能，用于对 OpenWrt 系统功能进行配置，在 OpenWrt 源码根目录下输入 make menuconfig 命令即可进入系统配置界面：



其中：

- **Target System** 用于选择芯片类型，默认选择 MT7628 对应的 Ralink RT288x/RT3xxx。
- **Subtarget** 用于选择目标子类型，默认选择 MT7628 based boards。
- **Target Profile** 选择不同路由器版本的选择参数，一般默认即可。
- **Target Images** 选择采用何种文件系统，官方推荐 squashfs，因为这种格式就算配置乱了还可以恢复刷机后的出厂设置，二是压缩后节省空间。jffs2 格式搞乱了就只能重刷了。
- **Global build settings** 编译时的一些全局参数，这些参数与 Linux 内核或 GCC 编译器相关。
- **Advanced configuration options**，高级选项参数，用于 OpenWrt 开发人员。
- **Build the OpenWrt Image Builder**，Image Builder 是一个预编译环境，适用在不从源代码编译的情况创建自定义镜像文件。它下载预编译的软件包并集成在一个可擦写的镜像中。
- **Build the OpenWrt SDK**，产生 OpenWrt SDK 环境，要进行开发的话必须选上。
- **Package the OpenWrt based Toolchain**，编译出 OpenWrt 的交叉编译工具。
- **Image configuration**，编译好的固件所附带的参数信息。
- **Base system**，基本系统命令软件包。
- **Administration**，高级管理命令软件包。
- **Boot Loaders**，引导程序，开发板用 Uboot 引导，不用固件附带，所以这里无选项。
- **Development**，开发用工具包。
- **Extra Packages**，其他软件包。
- **Firmware**，选择固件中是否带某些其他外围芯片的固件。
- **Kernel modules**，内核编译参数，这里的参数都来自芯片厂商对内核的预设配置，无需更改。
- **Languages**，编程语言软件包。
- **Libraries**，库软件包。
- **LuCI**，OpenWrt 下大名鼎鼎的 LuCI 界面包。
- **Mail**，与电子邮件相关的软件包。
- **Multimedia**，多媒体软件包。
- **Network**，与网络相关的软件包。
- **Ralink**，Ralink APSoC wifi 驱动。

- **Sound**，与音频相关的软件包。
- **Utilities**，工具软件包。

在配置过程中，并不是所有的选项都要选上。一方面，如果全部选上的话编译时间可能要按天计算；另一方面，用来存放固件的 Flash 容量也不足以安装这么多东西。所以要根据实际要完成的功能、硬件电路所支持的功能进行选择。我们这里采用开发板厂商提供的 OpenWrt 源代码，则 `make menuconfig` 里面的配置大部分不需要更改，但需要注意如下配置要选中（M 是编译成模块，*才是编译进内核）：

- Base system → block-mount，用于加载 U 盘和 SD 卡
- LuCI → Collections → luci，用于安装 web 管理界面
- Kernel Modules → other modules → kmod-sdhci-mt7620，不要选中，否则会与路由器的 LAN2 和 LAN3 冲突，导致这两个网口不可使用
- Kernel Modules → Native Language Support → kmod-nls-cp437/iso8859-1/utf8，增加语言模块支持
- Network → File Transfer → vsftpd，集成 vsftpd 服务
- Network → File Transfer → aria2，Network → File Transfer → aria2 → Aria2 configuration → Enable bittorrent support/Enable metalink support，集成 Aria2 并支持 bt 和磁力链接
- LuCI → Applications → luci-app-aria2，集成 aria2 的 web 配置页面
- Utilities → alsa-utils，声卡支持
- Sound → madplay-alsa，音频播放器

3.2.5 OpenWrt 源码编译

在 OpenWrt 源码目录下输入“`make V=s`”命令即可启动 OpenWrt 源码编译，V=s 是用来生成编译信息的，方便查找出错原因。由于 OpenWrt 系统要下载各种各样的源文件安装包，编译 Linux 内核，生成交叉工具链、各种工具和文件系统，工作量非常大。接下来需要的就是漫长的等待，少则 2~3 小时，多则以天计，好在这一过程都是自动完成的。编译成功则会出现如下输出：

```
Signing package index...
make[2]: Leaving directory '/home/openwrt/openwrt'
make[1]: Leaving directory '/home/openwrt/openwrt'
```

相应地源代码目录下会出现新的目录。

```
openwrt@openwrt-VirtualBox:~/openwrt$ ls
bin          Config.in    feeds.conf.default  LICENSE      README      target
BSDmakefile dl           include           Makefile     rules.mk    tmp
build_dir   docs        key-build         my.config    scripts     toolchain
config      feeds      key-build.pub     package      staging_dir  tools
```

其中：

- **build_dir**，OpenWrt 在这个目录中展开各软件包，进行编译。
- **bin**，保存编译完成后的二进制文件，包括完整的 bin 文件，所有的 ipk 文件。
- **dl**，在编译过程中使用的很多软件，刚开始下载源码并没有包含，而是在编译过程中从其他服务器下载的,这里是统一的保存目录。
- **staging_dir**，用于保存在 build_dir 目录中编译完成的软件,所以这里也和 build_dir 有同样的子目录结构。在我们开发自己的 ipk 文件时，编译过程中，预处理头文件，链接动态库，静态库都是到这个子文件夹中。
- **tmp**，临时文件夹，在编译过程中，有大量中间临时文件需要保存，都是在这里。
- **logs**，保存编译过程中出错的信息，只有当编译出错了才会出现。

编译完成的 OpenWrt 固件“openwrt-ramips-mt7628-mt7628-squashfs-sysupgrade.bin”可以在“openwrt 源码目录/bin/ramips/”下找到。注意请妥善保存好该固件，这样系统被你们玩坏后还可以重新刷系统。

3.2.6 安装 OpenWrt SDK

在 OpenWrt 源代码目录下输入命令“sudo tar jxvf bin/ramips/OpenWrt-SDK-ramips-mt7628_gcc-4.8-linaro_uClibc-0.9.33.2.Linux-x86_64.tar.bz2 -C /opt”，按提示输入超级用户密码后，将 OpenWrt SDK 压缩包解压到 ubuntu 系统的/opt/目录下。执行 sudo vi /etc/bash.bashrc 或 sudo gedit /etc/bash.bashrc，在文件最后添加：

```
export PATH=/opt/OpenWrt-SDK-ramips-mt7628_gcc-4.8-linaro_uClibc-0.9.33.2.Linux-x86_64/staging_dir/
toolchain-mipsel_24kec+dsp_gcc-4.8-linaro_uClibc-0.9.33.2/bin:$PATH

export PATH=/opt/OpenWrt-SDK-ramips-mt7628_gcc-4.8-linaro_uClibc-0.9.33.2.Linux-x86_64/staging_dir/
host/bin:$PATH

export STAGING_DIR=/home/openwrt/staging_dir
```

注意：增加这几句的目的是为了告诉操作系统在上述目录中寻找相应的命令；PATH 里的 /opt/ OpenWrt-SDK-ramips-mt7628_gcc-4.8-linaro_uClibc-0.9.33.2.Linux-x86_64 即为上边步骤中解压的目录；上面这个 STAGING_DIR 变量中的 openwrt 是放置 openwrt 源码的根目录，需要根据实际目录名做相应修改；这 3 个环境变量的名字和目录一定不要出错，否则会

导致 `bashrc` 奔溃，所有的命令都会找不到；万一出错，可以用绝对路径运行命令修改 `bash.bashrc` 文件后运行 `source /etc/bash.bashrc` 恢复。

保存退出后在终端执行命令 `source /etc/bash.bashrc` 使路径设置生效，并通过命令“`mipsel-openwrt-linux-gcc -v`”和“`touch --help`”查看命令执行是否正常。如果命令执行不成功，请检查“`mipsel-openwrt-linux-gcc`”这个命令文件是否已放在上述指定的目录中。

3.3 刷新 OpenWrt 固件

3.3.1 用 OpenWrt 升级页面烧写

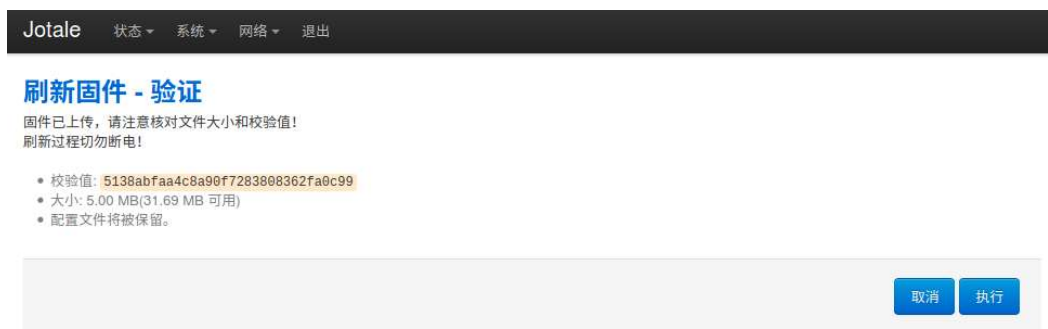
这种方法适用于 OpenWrt 系统可以正常启动的情况，如果 OpenWrt 系统无法正常启动，请采用 3.3.2 和 3.3.3 的方法。

- 1) 开发板上电，至启动完成。
- 2) 登录开发板地址“192.168.1.251”（可以用网口连接，也可以用 wifi 连接登录，默认用户名和密码均为 root）。
- 3) 点击系统配置页面的顶部菜单“系统”->“备份升级”；点击“选择文件”按钮在弹出页面选择要更新的固件（openwrt 源码目录 `/bin/ramips/` openwrt-ramips-mt7628-mt7628-squashfs-sysupgrade.bin）；根据需要选择是否“保留配置”。



- 4) 然后点击“刷写固件”按钮，如果固件正确，点击“执行”按钮开始烧写固件，完成后

系统会自动重启。



3.3.2 用 uboot 升级页面烧写

这种方式就是网上常说的“不死 uboot”网页烧写方式，可以用于升级“openwrt 固件”、“uboot 镜像”、“eeprom 数据”，烧写方式如下：

- 1) 将 PC 用网线连接开发板任意网口。
- 2) 开发板上电前，按下“RESET_FN_KEY”不放。
- 3) 开发板上电，“WLAN”灯亮起，接着快闪几次，说明已经进入 Uboot 网页升级页面状态，随后可以放开“RESET_FN_KEY”按键。
- 4) 通过开发板网口登录 uboot 网页升级页面“192.168.1.8”（注意：因为板子进入 Uboot 状态后，没有 DHCP 服务器功能，需要手动设置电脑 IP 为 192.168.1.*网段，比如 192.168.1.123。否则如果电脑与开发板的网段不同，可能会导致无法登录这个 IP 地址）



- 5) 在页面中选择正确的 openwrt 固件（openwrt 源码目录/bin/ramips/ openwrt-ramips-

mt7628-mt7628-squashfs-sysupgrade.bin)，点击上传按钮即可完成固件刷新。烧录完成后系统会自动重启。

在系统上电后，uboot 启动倒数计时结束前，在调试终端输入“6”，一样能使系统进入 uboot 升级页面状态。

3.3.3 用 uboot+串口+网络烧写

这是传统的嵌入式 Linux 开发中 uboot 比较通用的烧写方式，同样可用于升级“openwrt 固件”、“uboot 镜像”、“eeprom 数据”，不过比“用 uboot 升级页面烧写”的方式繁琐。具体步骤可参考《JS76x8 开发板使用手册 v1.9》中的 3.3 “用 uboot+串口+网络烧写”。

3.4 调试串口连接

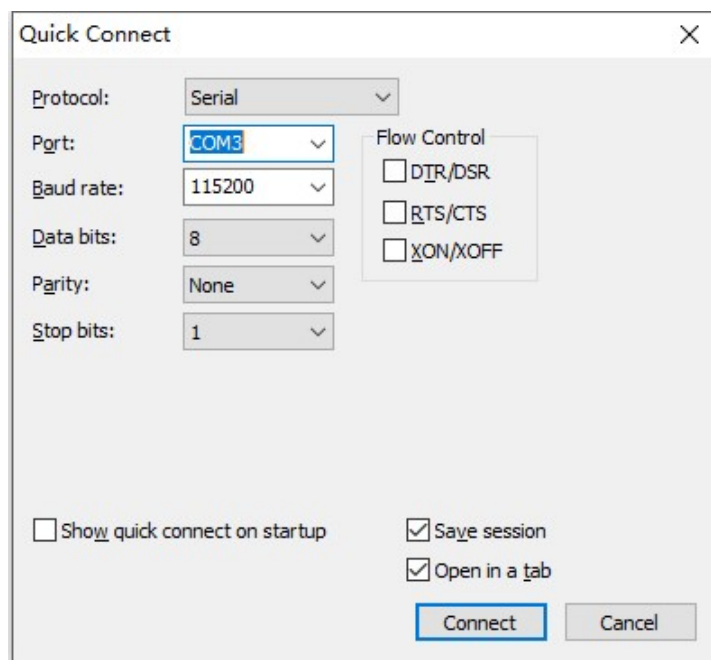
为了实现串口调试，首先需要安装串口调试软件。我们可以选择功能比较强大的 SecureCRT，它支持 SSH1、SSH2、Telnet、Rlogin、Serial 和 TAPI 协议；也可以选择更为精简的 Putty，无需安装即可使用。软件安装包在 ftp 资源下载目录下载，根据提示进行安装使用。

由于现在的计算机都取消了串口，开发板通过 CP2104 桥接芯片将调试串口转换成 USB 接口。用开发板提供的 USB 线一端连接开发板的 Micro USB 口，另一端连接计算机 USB 口，如果不能正确识别串口，则需要到 ftp 的资源下载目录找到 CP210x_Windows_Driver.zip 驱动进行安装。电脑正确识别后，在计算机的“设备管理器”->“端口”下会出现如下设备。

注意：图中“COM”是串口号，不同的计算机串口号可能会有所不同，请自行查看。

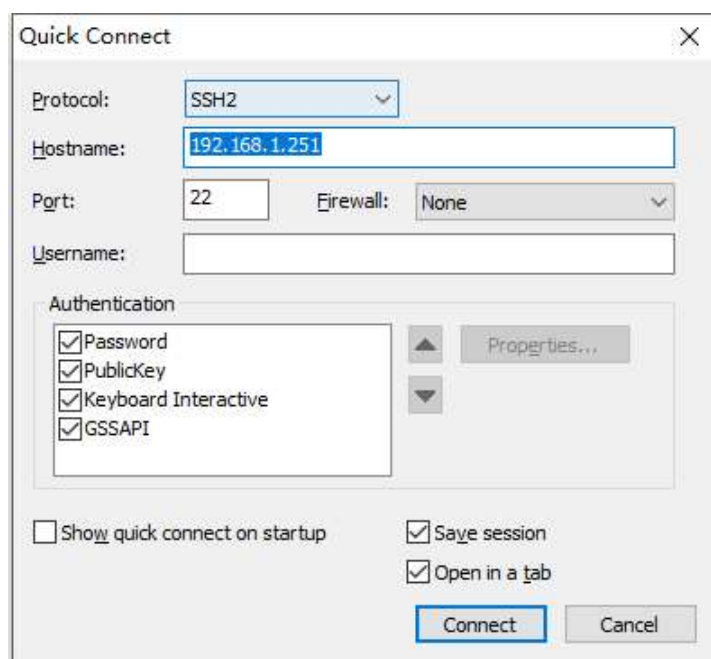


打开 SecureCRT，在“Quick Connect”弹出菜单中 Protocol 选择 Serial，Port 选择设备管理器显示的串口号，Band rate 选择 115200，其他默认。点击“Connect”连接到开发板。将 5V 电源连接到开发板，SecureCRT 即会显示相应的启动信息。启动完成后按回车键，此时 SecureCRT 即为 OpenWrt 的一个调试终端，其操作与 ubuntu 的命令行终端类似。



3.5 SSH 远程登录

除了用调试串口登录开发板的命令行终端，因为 OpenWrt 默认带了 SSH 服务器，还可以通过 SSH 网络登录开发板命令行终端。在登录前需要确保开发板和电脑网络已连接（可通过网线连接或无线连接，计算机开启 DHCP 即可）。在 Windows 下可以打开 SecureCRT，Protocol 选择 SSH2，Host name 填入开发板地址“192.168.1.251”，点击“Connect”后根据提示输入用户名和密码（默认都是 root）即可登录。在 Linux 下用命令行终端输入 `ssh root@192.168.1.251` 后按提示用密码登录。SSH 终端用法和调试串口终端的用法完全一致。



3.6 开发板与计算机的文件传输

3.6.1 用 SCP 协议传输文件

SCP 是 Linux 系统下基于 SSH 登录进行安全的远程文件拷贝的命令，如：

- `scp /home/hello root@192.168.1.251:/tmp`，将宿主机/home 目录下的 hello 文件复制到开发板/tmp 目录下，192.168.1.251 为开发板地址，root 为开发板登录的用户名，根据提示输入登录密码即可实现文件拷贝。与 `cp` 命令类似，`-r` 参数可用于文件夹的拷贝。
- `scp root@192.168.1.251:/tmp/hello /home`，将开发板/tmp 目录下的 hello 文件拷贝到宿主机的/home 目录。

在 Windows 下也可以用 WinSCP 传输文件，具体操作见《JS76x8 开发板 openwrt 入门教程》第 3.4.1.1 节，WinSCP 安装包见 FTP 资源下载目录。

3.6.2 用 NFS 服务传输文件

NFS (Network File System) 是一种在 linux 里面支持的比较好的网络文件系统。在 linux 里面，用户可以通过 NFS 客户端透明地读写位于远端 NFS 服务器上的文件，就像访问本地文件一样，非常方便。具体使用参见《JS76x8 开发板 openwrt 入门教程》第 3.4.1.2 节。

3.6.3 用 FTP 传输文件

FTP 是 File Transfer Protocol (文件传输协议) 的英文简称，而中文简称为“文传协议”，常用于 Internet 上的控制文件的双向传输。具体使用参见《JS76x8 开发板 openwrt 入门教程》第 3.4.1.3 节。

3.7 连接外网

开发板 PORT0、PORT1、PORT2 这三个网口默认均为 LAN 口，如果要上连接外网，则需要将其中某个网口切换为 WAN 口功能。要实现“WAN/LAN”功能切换，要利用 MT7268 自带的“交换机功能”。用地址“192.168.1.251”登录开发板配置页面（用户名和密码均为 root），打开“网络”->“交换机”菜单可以看到开发板默认有 2 个“VLAN ID”，其中“VLAN

ID 1” 连接到 LAN，“VLAN ID 2” 连接到 WAN。切换这些接口和 VLAN 的关联性即可实现这些接口“WAN/LAN”功能切换。如将端口 0 切换为 WAN 口，只需要将端口 0 和“VLAN ID 1”的关联性设为“关”，和“VLAN ID 2”的关联性设为“不关联”即可。



将机房计算机的网线拔出插到设置好 WAN 口，WAN 口默认采用 DHCP 获得 IP 地址，查看“网络”->“接口”页面 WAN 口的连接状态，如果连接不成功，则点击“修改”按钮，在弹出页面中选择静态地址，并将地址设置成与机房计算机一致。机房计算机如果要上网，可以用开发板提供的网线将计算机网口与开发板 LAN 口连接，并将计算机 IP 获取方式改为 DHCP。

4. OpenWrt 基础

4.1 软件包安装

4.1.1 通过 OPKG 在线安装

OPKG (Open/OpenWrt Package) 是一个轻量快速的软件包管理系统，是 IPKG 的克隆，目前已成为开源嵌入式系统领域的事实标准。OPKG 常用于路由、交换机等嵌入式设备中，用来管理软件包的下载、安装、升级、卸载和查询，并处理软件包的依赖关系。功能和桌面

Linux 操作系统 Ubuntu 中的 apt-get、Redhat 中的 yum 类似。

当执行“opkg update”命令进行软件列表的更新时，OPKG 首先会读取配置文件 /etc/opkg.conf，这个文件保存了 OPKG 的全局配置信息。紧接着 OPKG 会根据配置地址位置下载软件包列表文件 Packages.gz 到 /var/opkg-list 目录下，这个文件是软件仓库中所有软件列表及其依赖关系的清单。其后任何安装命令均需首先读取这两个文件。

软件安装后信息会保存在目录 /usr/lib/opkg/ 下面，这相当于 Windows 的注册表。它包含状态文件，OPKG 通过访问这个状态文件确定该软件是否已经安装、安装的版本，以及依赖关系是否满足等，从而可以确定安装软件的版本、文件路径等信息。

OPKG 命令执行会读取以下 3 部分信息：配置文件（/etc/opkg.conf）、已安装软件包信息（保存在 /usr/lib/opkg 下）和软件仓库的软件包信息（保存在 /var/opkg-lists 下）。具体命令如下：

- opkg update，更新可以安装的软件包列表，OPKG 在安装或升级时需要读取这个文件，因此**安装或升级前需要执行一次该命令**。
- opkg install <pkg>，安装软件包，pkg 为软件包名称，如果软件包之间有依赖关系，会自动下载所有被依赖的软件包，并依次将所有被依赖的软件包安装上。
- opkg remove <pkg>，卸载软件包，pkg 为软件包名称。需要注意的是，在安装时自动安装的软件包并不会删除，需要自己手动删除，或者在卸载软件包的同时增加（--autoremove）参数将不需要的安装包也删除。
- opkg upgrade <pkg1 pkg2 ...>，升级软件包，如果软件包没有安装，该命令执行之后和 opkg install 效果相同。如果升级多个软件包，以空格分隔列在命令之后即可。
- opkg list，用于列出所有可使用的软件包。
- opkg list-installed，用于列出已经安装的软件包。
- opkg list-changed-conffiles，用于列出用户修改过的配置文件。
- opkg files <pkg>，用于列出数据软件包 pkg 中的所有文件，这个软件包必须已经安装。
- opkg search <file>，用于列出提供 file 的软件包，注意需要传递文件的绝对路径。
- opkg find <regex>，用于列出软件包名称和正则表达式<regex>匹配的软件包。
- opkg info <pkg>，用于显示已安装软件包 pkg 的信息，包括名称、版本、所依赖的软件包名称、安装状态、安装时间等。如果没有指定参数则输出所有已安装软件包的信息。
- opkg download <pkg>，将软件包 pkg 下载到当前目录。

- `opkg print-architecture`，列出安装包的架构。
- `opkg whatdepends <-A> <pkg>`，输出依赖软件包 `pkg` 的软件包。

4.1.2 通过 OpenWrt 配置页面安装

用地址“192.168.1.251”登录开发板配置页面（用户名和密码均为 root），在“系统”->“软件包”页面下进行软件包的查看、安装与卸载。

Jotale 状态 ▾ 系统 ▾ 网络 ▾ 退出

软件包

动作 配置

空闲空间: 97% (26.07 MB)

下载并安装软件包: OK

过滤器: 查找软件包

状态

已安装软件包 可用软件包

	软件包名称	版本
移除	alsa-lib	1.0.28-1
移除	alsa-utils	1.0.28-2
移除	base-files	157.2-r49396
移除	busybox	1.23.2-1
移除	dnsmasq	2.73-1

4.1.3 离线安装

可以到“https://downloads.openwrt.org/chaos_calmer/15.05.1/ramips/mt7628/packages/”或者到“ftp 资源下载目录/[IPK 安装包/packages](#)”目录下找到需要安装的包，然后通过 3.6 介绍的电脑和开发板文件互传方法将相应的 ipk 安装包拷贝到开发板“/tmp”目录下，在调试中断执行命令：`opkg install /tmp/安装包名称.ipk`，如果提示要先安装其他的安装包，可以用同样的方法找到依赖包上传开发板用 `opkg` 命令进行安装。

4.2 UCI 配置命令

4.2.1 UCI 简介

UCI (Unified Configuration Interface) 是集中式配置信息管理接口的缩写, 它是 OpenWrt 引进的一套配置参数管理系统。UCI 管理了 OpenWrt 下最主要的系统配置参数, 并且提供了简单、容易、标准化的人机交互接口。UCI 中已经包含了网络配置、无线配置、系统信息配置等基本路由器所需要的主要配置参数。同时, UCI 也可以帮助开发人员快速地建立一套基于 OpenWrt 的智能路由产品控制界面。

UCI 的配置文件全部存储在 /etc/config 目录下, 由于 UCI 的数据文件较为简单, 并且具备了很好的直观感, 所以配置文件既可以使用 UCI 命令进行修改, 也可以使用文本编辑器直接修改。但如果两种方式都使用, 需要注意 UCI 命令修改会产生缓存, 每次修改好要尽快确认保存, 以免出现冲突。

4.2.2 UCI 配置文件语法

配置文件由配置节 (section) 组成, 配置节由多个 “name/values” 选项组成。每一个配置节都需要有一个类型标识, 但不一定需要名称。每一个选项都有名称和值, 写在其所属的配置节中。

```
config system
    option hostname OpenWrt
    option timezone UTC
config timeserver ntp
    list server 0.openwrt.pool.ntp.org
    list server 1.openwrt.pool.ntp.org
    list server 2.openwrt.pool.ntp.org
    list server 3.openwrt.pool.ntp.org
    option enabled 1
    option enable_server 0
```

上述配置文件中 “config system” 语句定义了一个配置节的开始, 配置类型为 “system” 但没有名称, 没有名称的配置节称为匿名配置节。选项 “option hostname OpenWrt” 和 “option timezone UTC” 两行定义了这个配置节的两个简单配置。类型和选项的含义均由应用程序来决定, 类型一般用于应用程序决定如何处理配置节包含的配置选项。如果一个选项

是不存在的并且是必需的，那应用程序通常会触发一个异常或者记录一个异常的日志，然后程序退出。

“config timeserver ntp”语句定义了另外一个配置节的开始，类型为“timeserver”，名称为“ntp”。在开始带有 list 关键字的选项，标识有多个值被定义，所有的语句有同一个选项名称“server”。

4.2.3 统一配置原理

OpenWrt 有很多独立的第三方应用程序，大多数应用程序的软件包维护者已经制作了 UCI 兼容的配置文件，启动时由 UCI 配置文件转换成软件包的原始配置文件。这是在运行初始化脚本/etc/init.d/中执行的。例如，在 DNS 代理服务器 dnsmasq 进程启动的情况下，文件/var/etc/dnsmasq.conf 是从 UCI 配置文件/etc/config/dhcp 生成并覆盖的，是运行/etc/init.d/dnsmasq 脚本进行配置文件转换的。因为应用程序的配置文件是启动时通过 UCI 转换生成的，因此它不需要存储在非易失性存储器中，通常存储在内存中，而 var 目录为其内容在正常运行时不断变化的目录，因此将 var 目录创建为/tmp 目录的一个链接。OpenWrt 的大多数配置都是基于 UCI 文件的，如果想在软件原始配置文件调整设置，可以通过禁用 UCI 方法来实现。

OpenWrt 系统的核心配置分成很多个文件，并且都位于/etc/config 目录下，每个文件涉及系统配置的某一部分。你可以用文本编辑器修改，或用命令行实用程序 UCI 编辑配置文件。UCI 配置文件也可以通过各种编程 API（如 shell、Lua 和 C 等）来修改，这也是 Web 接口例如 LuCI 修改 UCI 文件的方式。无论采用哪种方式修改 UCI 配置文件后，受影响的服务或可执行程序必须由 init.d 进行重启，这样更新的 UCI 配置才会真正生效。许多程序是通过它们的初始化脚本与 UCI 配置兼容的，init.d 将 UCI 配置转换为它们软件特定的配置文件，首先在该软件预期的位置生成一个配置文件，它通过重新启动可执行程序再次读入配置。注意：如果只是直接启动可执行文件，没有通过 init.d 调用，将不会将一个 UCI 配置文件更新到特定程序响应的配置文件位置，在/etc/config/的修改将不会对现有进程有任何影响。

4.2.4 UCI 命令

在开发调整配置的时，可以直接使用文本编辑器修改 UCI 配置文件，但这种方式比较低效。UCI 统一配置文件的目的是所有 OpenWrt 配置可以通过统一接口读取和修改，UCI 实

用工具提供了修改和分析 UCI 文件的脚本编程开发接口，其具体用法为：

```
#uci    [<option>]  <command> [<arguments>]
```

UCI 读取类语法有：

- 取得节点类型：uci get <config>.<section>，如 uci get system.ntp
- 取得一个值：uci get <config>.<section>.<option>，如 uci get system.ntp.server
- 显示全部 UCI 配置：uci show
- 显示指定文件配置：uci show <config>，如 uci show system
- 显示指定节点名字配置：uci show <config>.<section>，如 uci show system.ntp
- 显示指定选项配置：uci show <config>.<section>.<option>，如 uci show system.ntp.server
- 显示尚未生效的修改记录：uci changes <config>，如 uci changes system
- 匿名节点显示：uci show -X <config>.<section>.<option>

UCI 写入类语法有：

- 在文件中增加一个匿名节点：uci add <config> <section-type>
- 在文件中增加一个节点：uci set <config>.<section>=<section-type>
- 在节点中增加一个选项和值：uci set <config>.<section>.<option>=<value>
- 在列表中增加一个值：uci add_list <config>.<section>.<option>=<value>
- 修改一个节点类型：uci set <config>.<section>=<section-type>
- 修改一个选项的值：uci set <config>.<section>.<option>=<value>
- 删除指定名字的节点：uci delete <config>.<section>
- 删除指定选项：uci delete <config>.<section>.<option>
- 删除列表：uci delete <config>.<section>.<list>
- 删除列表中的一个值：uci delete_list <config>.<section>.<option>=<string>
- 修改生效：uci commit <config>，注意任何写入类的语法，最终都要执行修改生效命令，否则所做修改只在缓冲中不会生效，切记！

4.2.5 UCI API 接口

UCI 接口命令非常规范，统一以小写的 uci 开头并放在 uci.h 头文件中。大多数函数的第一个参数均为 uci_context 的指针变量，这个变量在程序初始化时调用 uci_alloc_context 函

数分配空间并设置初始值。在程序执行结束时调用 `uci_free_context` 函数释放空间。相应的 API 函数有：

- `uci_alloc_context`，分配 UCI 上下文环境对象
- `uci_free_context`，释放 UCI 上下文环境对象
- `uci_load`，解析 UCI 配置文件，并存储到 UCI 对象中
- `uci_unload`，从 UCI 上下环境对象中 unload 配置文件
- `uci_lookup_ptr`，分隔字符串并查找
- `uci_set`，设置元素值，如果必要将创建一个元素
- `uci_delete`，删除一个元素、配置节或选项
- `uci_save`，保存一个 package 修改的 delta
- `uci_commit`，提交更改 package，提交将重新加载整个 `uci_package`
- `uci_set_confdir`，修改 UCI 配置文件的存储位置，默认为 `/etc/config`

4.3 软件包开发

以下以 `mytest` 工程为例讲述 `ipk` 软件包开发的全过程。`mytest` 软件包通过串口以固定的时间间隔发送字符串消息，发送间隔和发送的消息内容可以通过 OpenWrt 的 Web 配置界面进行修改。

4.3.1 软件包工程建立

在 OpenWrt 源代码目录下建立 `mypackages/examples` 目录用于存放我们自己开发软件包工程，具体命令为：`mkdir /OpenWrt 源代码根目录/mypackages`，`mkdir /OpenWrt 源代码根目录/mypackages/examples`。在 `mypackages/examples` 目录下建立 `mytest` 目录用于保存 `mytest` 工程（**注意：目录名称要与工程名称一致**），在 `mytest` 目录下建立 `src` 目录用于存放源代码，在 `src` 目录下建立 `mytest.c` 文件，其内容如下：

```
#include <stdio.h>

int main(void)
{
    printf("\nHello, world!\n\n");
    return 0;
}
```

在 mytest 目录下建立 Makefile 文件，并输入如下内容：

```
include $(TOPDIR)/rules.mk

PKG_NAME:=mytest
PKG_VERSION:=1.0
PKG_RELEASE:=1
SOURCE_DIR:=$(shell pwd)/src

include $(INCLUDE_DIR)/package.mk

define Package/mytest
    SECTION:=examples
    CATEGORY:= Examples
    TITLE:=A test project!
endef

define Package/mytest/description
    This is a test project.
endef

define Build/Prepare
    mkdir -p $(PKG_BUILD_DIR)
    cp $(SOURCE_DIR)/* $(PKG_BUILD_DIR)
    $(Build/Patch)
endef

define Build/Compile
    $(TARGET_CC)    $(TARGET_CFLAGS)    -o    $(PKG_BUILD_DIR)/mytest.o    -c
$(PKG_BUILD_DIR)/mytest.c
    $(TARGET_CC)    $(TARGET_LDFLAGS)    -o    $(PKG_BUILD_DIR)/$1
$(PKG_BUILD_DIR)/mytest.o
endef

define Package/mytest/install
    $(INSTALL_DIR) $(1)/usr/bin
    $(INSTALL_BIN) $(PKG_BUILD_DIR)/mytest $(1)/usr/bin
endef
$(eval $(call BuildPackage,mytest))
```

其中：TOPDIR 是 OpenWrt SDK 的路径，rules.mk 定义了 Makefile 中使用的一些通用变量和函数；**PKG_NAME** 为软件包名字，和文件夹名字一致；PKG_VERSION 为软件包版本，PKG_RELEASE 为 Makefile 版本；SOURCE_DIR 为源代码目录，\$(shell pwd)为 Makefile 所在

目录；SECTION 为包的种类，CATEGORY 指定包出现在 make menuconfig 的哪个目录下，TITLE 是对包的简单描述；Package/mytest/description 是对包的详细描述；define Build/Prepare 定义如何准备编译本软件包，创建编译目录，拷贝源代码，最后一行出于兼容性考虑是必须的；define Build/Compile 是对软件包的编译指令，这里需要指令编译的 C 源文件和目标文件；PKG_BUILD_DIR 是包编译的目录；INSTALL_DIR 是包安装目录；\$1 表示传入的第一个参数。

建立完目录和文件后，我们还需要将软件包集成到 OpenWrt 源码中，具体步骤如下：

- 在 OpenWrt 源代码根目录 feeds.conf.default 文件的末尾添加：

```
src-link mypackages OpenWrt 源代码根目录/mypackages
```

注意 OpenWrt 源代码根目录需要替换成自己真实的目录。

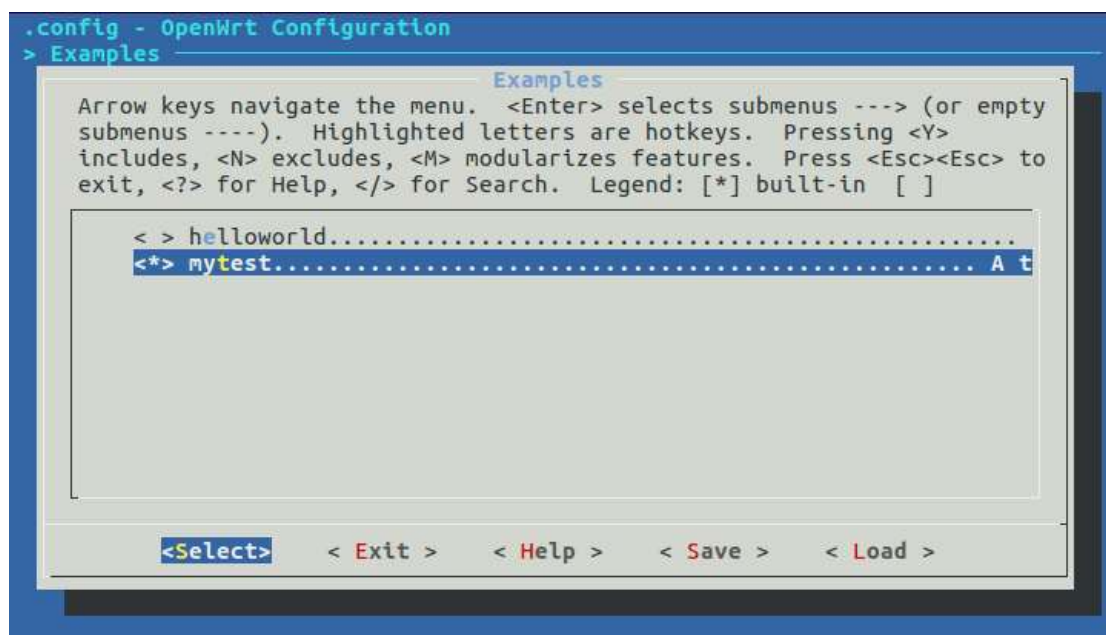
- 更新并安装 feeds，具体命令为：

```
cd OpenWrt 源代码根目录
```

```
./scripts/feeds update -a
```

```
./scripts/feeds install -a
```

- 在 OpenWrt 源代码根目录下输入 make menuconfig 命令，在弹出的窗口里查看 mytest 工程是否添加成功，如果成功的话，弹出的窗口有“Examples”选项，按回车进入后会有 mytest 选项，选中该选项并保存退出。注意：如果不选择该选项，软件包将不会被编译，选 M 是编译模块，选*是编译模块并安装到内核。



- 在 OpenWrt 源代码根目录下输入命令 “make package/mytest/compile V=s” 对软件包

进行编译，编译成功则可在“源代码根目录/bin/rampis/packages/mypackages”目录下找到 mytest_*_ramips_24kec.ipk (*为通配符，不同的包版本号*也会有不同)，该文件即为 ipk 安装包。

- 进入“源代码根目录/bin/rampis/packages/mypackages”目录，输入命令“scp mytest_*_ramips_24kec.ipk root@192.168.1.251:/tmp”将 ipk 安装包拷贝到开发板的 /tmp 目录下。
- 用 SecureCRT 登录开发板，进入 tmp 目录，执行命令“opkg install /tmp/mytest_*_ramips_24kec.ipk”安装软件包后，输入命令“mytest”即可输出程序运行结果。

```
root@Jotale:/tmp# opkg install /tmp/mytest_1.0-3_ramips_24kec.ipk
Installing mytest (1.0-3) to root...
Configuring mytest.
root@Jotale:/tmp# mytest

Hello, openwrt!

root@Jotale:/tmp#
```

4.3.2 读取软件包配置

根据软件包的功能需求，软件包需要改变的配置有两个：一个是串口发送字符串的时间间隔，另一个是串口发送字符串的具体内容。我们将这两个配置在 UCI 配置文件 mytest.config 中定义（UCI 相关知识详见章节 4.2），其中 period 为时间间隔，output 为串口输出内容。src 目录下 mytest.config 如下所示：

```
config mytest parameter
    option period 0
    option output 'Hello OpenWrt!'
```

软件包的配置一般在启动时读取，我们在 src 目录下新建 config.c 和对应的 config.h 文件实现配置信息的读取。其中 config.h 如下：

```
struct CONFIG
{
    char output[100]; //输出字符串
    unsigned int period; //时间间隔，单位为毫秒
};
int GetConfig(struct CONFIG *config);
```

config.c 中定义了两个函数。GetValue 用于从 UCI 配置文件中读取选项的值；GetConfig 供主函数调用，读取配置值。

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <uci.h>
#include "config.h"
int GetValue(struct uci_context *ctx, char *key, char *value, int n)
{
    char strKey[100];
    struct uci_ptr ptr;
    snprintf(strKey, sizeof(strKey), "mytest.parameter.%s",key);//组成完整的选项名
    if(uci_lookup_ptr(ctx, &ptr, strKey, true) == UCI_OK)//查找选项
    {
        strncpy(value,ptr.o->v.string, n-1);//复制选项的值
        return 1;
    }
    return 0;
}
int GetConfig(struct CONFIG *config)
{
    struct uci_context *ctx = uci_alloc_context();//获得 UCI 上下文环境对象
    if(!ctx)    //出错
    {
        printf("UCI alloc error\n");
        return -1;
    }
    char period[20];
    if(GetValue(ctx, "period", period, sizeof(period)))
    {
        config->period = atoi(period);
    }
    else
    {
        uci_free_context(ctx);
        return -2;
    }
    if(GetValue(ctx,"output", config->output, sizeof(config->output))==0)
    {
        uci_free_context(ctx);
        return -3;
    }
    uci_free_context(ctx);
    return strlen(config->output);
}
```

在 mytest.c 中，主函数调用 GetConfig 函数在初始化时读取软件包配置，源代码修改为：

```
#include <stdio.h>
#include "config.h"

int main(void)
{
    struct CONFIG config;
    int rtn = GetConfig(&config);
    if(rtn<0)
    {
        printf("Package configuration error: %d\n",rtn);
        return 0;
    }
    printf("Output = %s\n",config.output);
    printf("Period = %d ms\n", config.period);
    return 0;
}
```

由于增加了 config.c 和 config.h 两个源文件，相应地 Makefile 也需要相应地修改。在 config.c 中我们调用了 UCI API 相关的函数，UCI 库的头文件安装在 /usr/local/include 目录下，动态链接库安装在 /usr/local/lib/libuci.so，为了使用这个动态链接库，我们需要在 Makefile 的“define Package/mytest”、“define Build/Compile”、“define Package/mytest/install”进行修改，相应的修改内容如下：

```
define Package/mytest
    SECTION:=examples
    CATEGORY:=Examples
    TITLE:=A test project!
    DEPENDS:=+libuci
endef

define Build/Compile
    $(TARGET_CC) $(TARGET_CFLAGS) -o $(PKG_BUILD_DIR)/mytest.o -c
$(PKG_BUILD_DIR)/mytest.c
    $(TARGET_CC) $(TARGET_CFLAGS) -o $(PKG_BUILD_DIR)/config.o -c
$(PKG_BUILD_DIR)/config.c
    $(TARGET_CC) $(TARGET_LDFLAGS) -o $(PKG_BUILD_DIR)/$1
$(PKG_BUILD_DIR)/mytest.o $(PKG_BUILD_DIR)/config.o -luci
endef

define Package/mytest/install
    $(INSTALL_DIR) $(1)/usr/bin
    $(INSTALL_BIN) $(PKG_BUILD_DIR)/mytest $(1)/usr/bin
    $(INSTALL_DIR) $(1)/etc/config
    $(INSTALL_CONF) ./src/mytest.config $(1)/etc/config/mytest
endef
```


按照 4.3.1 中的软件包编译、上传、安装与运行步骤在开发板上查看运行结果如下：

```
root@jotale:/etc/config# opkg install /tmp/mytest_1.0-3_ramips_24kec.ipk
Installing mytest (1.0-3) to root...
Configuring mytest.
root@jotale:/etc/config# mytest
Output = Hello openwrt!
Period = 0 ms
```

4.3.3 通过 LuCI 修改软件包配置

接下来我们实现通过 OpenWrt 的 LuCI 配置页面修改软件包配置的功能。LuCI(Lua+UCI) 是 OpenWrt 上的 Web 管理界面，它采用了 MVC 三层架构，使用 Lua 脚本开发，所以开发 LuCI 的配置界面基本不需要编辑 html 代码。要实现一个 LuCI 界面，至少需要如下三个文件：

A: /etc/config/mytest，即软件包配置文件 mytest.config 在包安装时拷贝进去的；

B: /usr/lib/lua/luci/controller/other.lua

C: /usr/lib/lua/luci/model/cbi/mytest.lua

A 是需要修改的软件包 UCI 配置文件，是包安装时 mytest.config 的拷贝，可以使用 shell 的 uci 命令进行读写。在我们的实例中，A 中有一个 mytest 类型的 parameter 节，该节有 delay 和 output 两个选项分别控制软件包发送消息的时间间隔和内容。

B 的作用是告诉系统你自定义界面的显示位置，相邻排序，并指向 C 文件。B 文件内容如下：

```
1 module("luci.controller.other",package.seeall)
2
3 function index()
4     entry({"admin", "other"}, {}, _("其他"), 60).index = true
5     if nixio.fs.access("/etc/config/mytest") then
6         entry({"admin", "other", "mytest"}, cbi("mytest"), _("测试"), 1)
7     end
8 end
```

第 1 行是固定模式，对应“/usr/lib/lua/luci/controller/other.lua”。

第 4 行中，entry(path, target, title=nil, order=nil)添加一个新的模块入口；path 指访问路径，这里{“admin”，“other”}表示在以管理员身份登录页面才可以看到，other 表示一级菜单名；target 表示调用的功能，这里不需要调用，因此为空；title 指显示名称；order 指显示顺序，LuCI 会对每个模块的 order 进行排序确定显示位置，这里填 60 是为了把它放在最后。

第 5 行判断 UCI 配置文件 A 是否存在。

第 6 行定义了“其他”一级菜单下的选项，该选项名称为“测试”，点击后指向“/usr/lib/lua/luci/model/cbi/mytest.lua”即 C 文件。

通过“scp other.lua root@192.168.1.251:/usr/lib/lua/luci/controller”命令将“other.lua”文件上传到开发板指定位置。然后先后在开发板调试界面输入“uci set luci.ccache.enable=0”和“uci commit luci”将 luci 的缓存功能去掉；这样 LuCI 每次都是直接读取 Flash 上文件而不是访问内存中的缓存，other.lua 只要上传到开发板指定位置即可直接生效，可以方便调试；缺点是 Flash 上的文件访问效率较低，调试完成后需要将 luci.ccache.enable 设为 1 提升效率。在浏览器中输入网址“192.168.1.251”，登录后即可看到如下页面：



此时点击“测试”选项将会出错，这是因为点击后所指向的文件“/usr/lib/lua/luci/model/cbi/mytest.lua”并不存在，接下来我们新建 mytest.lua 文件，其内容如下：

```
1 local fs = require "nixio.fs"
2
3 m = Map("mytest", translate("测试"), translate("测试页面修改mytest软件包配置
  中串口发送的消息和发送时间间隔。"))
4
5 s = m:section(TypedSection, "mytest", "")
6 s.addremove = false
7 s.anonymous = true
8
9 view_period = s:option(Value, "period", translate("发送间隔"))
10 view_period.default = 0
11
12 view_output = s:option(TextValue, "output", "消息内容")
13 view_output.reempty = false
14 view_output.rows = 2
15
16 return m
```

第 1 行是引用依赖库，这个依赖库是用来读取和写入文件的；要实现读写配置文件的功能，这个库是必不可少的。

第 3 行是固定格式，“mytest”是 UCI 配置文件的文件名，“测试”是配置页面标题，最后一个为配置页面的描述。

第 5 行指定了配置文件的 section，我们要读写的是 mytest 类型的 section，这对应于 UCI 配置文件中的“config mytest parameter”一行。

第 6 行设定不允许增加或删除 section。

第 7 行设定是否显示 section 的名称。

第 9 行增加一个文本框控件，它对应于 UCI 配置文件中的“period”选项，“发送间隔”是文本框前的提示文字。

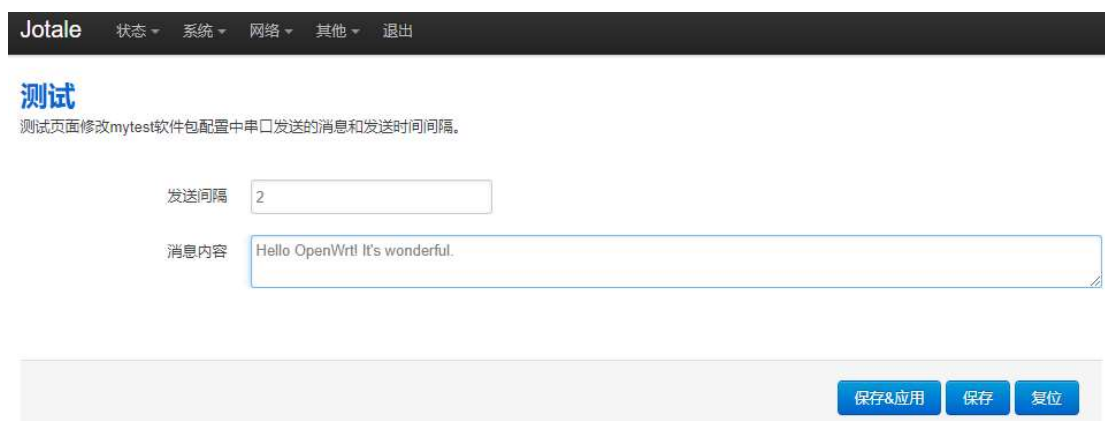
第 10 行设定文本框的默认值，如果从 UCI 配置文件读不到该选项，则默认为 0。

第 12 行增加一个大的文本输入框，它对应于 UCI 配置文件中的“output”选项，“消息内容”是提示文字。

第 13 行设定即使文本输入框为空，也不删除 UCI 配置文件中的“output”选项；如果该项设置为 true 且文本输入框为空，则提交表单后会将 UCI 配置文件中的“output”选项删除。

第 14 行设定本输入框的行数为 2 行。

通过“`scp mytest.lua root@192.168.1.251: /usr/lib/lua/luci/model/cbi`”命令将“mytest.lua”文件上传到开发板指定位置后，在 Web 配置页面点击“测试”选项，即可出现如下配置命令：



点击“保存&应用”即可修改相应配置，在开发板调试界面输入“mytest”命令可以看到修改的配置已经生效：

```
root@jotale:/etc/config# mytest
output = Hello openwrt! It's wonderful.
Period = 2 ms
root@jotale:/etc/config#
```

为了软件包安装时能自动将 other.lua 和 mytest.lua 这两个文件安装到开发板指定位置，我们首先将 other.lua 和 mytest.lua 这两个文件放到“OpenWrt 源代码目录/mypackages/examples/mytest/src”目录，并在 Makefile 的 Package/mytest/install 段增加命令安装这两个文件：

```

define Package/mytest/install
    $(INSTALL_DIR) $(1)/etc/config
    $(INSTALL_CONF) ./src/mytest.config $(1)/etc/config/mytest
    $(INSTALL_DIR) $(1)/usr/bin
    $(INSTALL_BIN) $(PKG_BUILD_DIR)/mytest $(1)/usr/bin
    $(INSTALL_DIR) $(1)/usr/lib/lua/luci/controller
    $(INSTALL_CONF) ./src/other.lua $(1)/usr/lib/lua/luci/controller/other.lua
    $(INSTALL_DIR) $(1)/usr/lib/lua/luci/model/cbi
    $(INSTALL_CONF) ./src/mytest.lua $(1)/usr/lib/lua/luci/model/cbi/mytest.lua
endef

```

重新编译软件包并上传到开发板，将开发板上 other.lua 和 mytest.lua 这两个文件删除后重新安装软件包，如果成功的话可以在相应目录看到 other.lua 和 mytest.lua 这两个文件。

关于 LuCI 的详细信息还可以查看 ftp 上的《openwrt luci 学习笔记》、《【OpenWRT 之旅】LuCI 探究》、“<https://blog.csdn.net/lvshaorong/article/details/53939138>”、“<https://github.com/openwrt/luci/wiki/CBI>”、“https://github.com/seamustuo/hy/luci_tutorials”或自己在网络上搜索“LuCI”相关的开发文档。

4.3.4 串口消息的定时输出

mytest 软件包每次启动时首先从 UCI 配置文件读取 period 和 output 两个参数，然后以 period 为周期定时通过串口发送 output 的内容。串口是计算机的一种常用接口，我们开发板上提供了一个 RS232 串口，对应 Linux 中的串口设备文件“/dev/ttyS1”。Linux 对设备的操作方法与对文件的操作方法是一样的，因此对串口的读写可以使用简单的“read”、“write”来完成，所不同的是只需要对串口的其他参数做配置。有关 Linux 下的串口编程可参考 ftp 上的《嵌入式 Linux 应用程序开发详解-第 6 章 文件 IO 编程》。新建 comport.c 和 comport.h 文件实现串口配置的函数 SetComPort，comport.h 代码如下：

```
int SetComPort(int fd, int nSpeed, int nBits, char nEvent, int nStop);
```

comport.c 代码如下：

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <termios.h>
#include <errno.h>

int SetComPort(int fd, int nSpeed, int nBits, char nEvent, int nStop)
{
    struct termios newtio, oldtio;

    //保存测试现有串口参数，如果串口号出错，会有相关的出错信息
    if(tcgetattr(fd, &oldtio) != 0)
    {
        perror("SetupSerial 1");
        return -1;
    }
}

```



```

bzero(&newtio, sizeof(newtio)); //设置清零

//设置字符大小
newtio.c_cflag |= CLOCAL | CREAD;           //本地连接和接收使能
newtio.c_cflag &= ~CSIZE;                   //清除设置字符大小的位隐码
switch(nBits)
{
    case 7:
        newtio.c_cflag |= CS7;
        break;
    case 8:
        newtio.c_cflag |= CS8;
        break;
}

//设置奇偶校验位
switch(nEvent)
{
    case 'O': //奇校验
        newtio.c_cflag |= PARENB;
        newtio.c_cflag |= PARODD;
        newtio.c_iflag |= (INPCK|ISTRIP);
        break;
    case 'E': //偶校验
        newtio.c_cflag |= PARENB;
        newtio.c_cflag &= ~PARODD;
        newtio.c_iflag |= (INPCK|ISTRIP);
        break;
    case 'N': //无奇偶校验位
        newtio.c_cflag &= ~PARENB;
        break;
}

//设置波特率, 输入输出都要设置
switch(nSpeed)
{
    case 2400:
        cfsetispeed(&newtio, B2400);
        cfsetospeed(&newtio, B2400);
        break;
    case 4800:
        cfsetispeed(&newtio, B4800);
        cfsetospeed(&newtio, B4800);
        break;
    case 9600:
        cfsetispeed(&newtio, B9600);
        cfsetospeed(&newtio, B9600);
        break;
    case 19200:
        cfsetispeed(&newtio, B19200);
        cfsetospeed(&newtio, B19200);
        break;
    case 38400:
        cfsetispeed(&newtio, B38400);
        cfsetospeed(&newtio, B38400);
        break;
    case 57600:
        cfsetispeed(&newtio, B57600);
        cfsetospeed(&newtio, B57600);
        break;
    case 115200:
        cfsetispeed(&newtio, B115200);
        cfsetospeed(&newtio, B115200);
        break;
    case 460800:
        cfsetispeed(&newtio, B460800);
        cfsetospeed(&newtio, B460800);
        break;
    default:
        cfsetispeed(&newtio, B9600);
        cfsetospeed(&newtio, B9600);
        break;
}

```

```

//设置停止位
if(nStop == 1)
    newtio.c_cflag &= ~CSTOPB;
else if(nStop == 2)
    newtio.c_cflag |= CSTOPB;

//设置读取每个字符的等待时间, 单位为100ms, 没有特别要求就设为0
newtio.c_cc[VTIME] = 0;
//设置最少读取的字符数
newtio.c_cc[VMIN] = 0;

//清空之前的未接收字符
tcflush(fd, TCIFLUSH);

//激活新配置
if((tcsetattr(fd, TCSANOW, &newtio)) != 0)
{
    perror("com set error");
    return -1;
}

return 0;
}

```

为了将 `comport.c` 编译进软件包, 我们还需要对 Makefile 的 “define Build/Compile” 段进行修改:

```

define Build/Compile
    $(TARGET_CC) $(TARGET_CFLAGS) -o $(PKG_BUILD_DIR)/mytest.o -c $(PKG_BUILD_DIR)/mytest.c
    $(TARGET_CC) $(TARGET_CFLAGS) -o $(PKG_BUILD_DIR)/config.o -c $(PKG_BUILD_DIR)/config.c
    $(TARGET_CC) $(TARGET_CFLAGS) -o $(PKG_BUILD_DIR)/comport.o -c $(PKG_BUILD_DIR)/comport.c
    $(TARGET_CC) $(TARGET_LDFLAGS) -o $(PKG_BUILD_DIR)/$1 $(PKG_BUILD_DIR)/mytest.o $(PKG_BUILD_DIR)/config.o -luci $(PKG_BUILD_DIR)/comport.o
endef

```

同时我们还需要修改 `mytest.c`, 增加串口发送相应的代码:

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <termios.h>
#include <errno.h>
#include "config.h"
#include "comport.h"

#define BANDRATE 115200
#define COMPORT_NUMBER "/dev/ttyS1"
int main(void)
{
    int fd;
    struct CONFIG config;

    //读取配置并判断配置合理性
    int rtn = GetConfig(&config);\
    if(rtn < 0)//读取配置失败
    {
        printf("Package configuration error: %d\n", rtn);
        return 0;
    }
    //判断发送间隔是否合理, 后面的条件以1个自己16位算(留余量)判断发送时间
    if((config.period <= 0) || (16000*rtn/BANDRATE >= config.period))
    {
        printf("Period error: %d ms\n", config.period);
        return 0;
    }
}

```

```

if(rtn == 0)//发送消息为空
{
    printf("No message to be send!\n");
    return 0;
}

//以阻塞读和写的方式打开并设置串口
if((fd = open(COMPORT_NUMBER, O_RDWR | O_NOCTTY)) < 0)
{
    printf("Open %s failed\n", COMPORT_NUMBER);
    return 0;
}
SetComPort(fd, 115200, 8, 'N', 1);//设置成8N1 115200bps
//循环发送数据
char tail[] = "\r\n";
while(1)
{
    write(fd, config.output, strlen(config.output)+1);
    write(fd, tail, 2);
    usleep(1000*config.period);
}

return 0;
}

```

至此源代码已准备完毕，在 OpenWrt 源代码目录下输入“make package/mytest/compile V=s”命令重新编译软件包；如果出错则根据出错信息修改源代码重新编译；如果成功则进入“源代码根目录 /bin/rampis/packages/mypackages”目录，输入命令“scp mytest*_ramips_24kec.ipk root@192.168.1.251:/tmp”将 ipk 安装包拷贝到开发板的/tmp目录下。

用 SecureCRT 登录开发板，进入 /tmp 目录，执行命令“opkg install /tmp/mytest*_ramips_24kec.ipk”安装软件包；登录开发板 Web 管理界面修改发送间隔和消息内容（注意：period 默认为 0，只有设置正确时间串口才会发送数据）；输入命令“mytest”运行软件；将 USB 转串口线连接到开发板的 RS232 串口，用 SecureCRT 打开对应串口（115200bps，8N1）即可看到串口输出的信息。通过开发板 Web 管理界面可改变发送间隔和消息内容，参数改变后软件包需重启方可使改变生效。



4.3.5 软件包的自动运行

OpenWrt 引入了一个便于控制的启动机制，这种机制是在/etc/rc.d 目录下创建每个软件的软链接方式，由 rcS 脚本在该目录读取启动命令的软链接，然后启动软链接所指向的程序，由于每一个软链接均包含一个数字，这样就可以按照数字从小到大的顺序读取并进行启动了。

每个需要自动启动的软件都需要一个与软件同名的 shell 脚本，该脚本位于“/etc/init.d”目录下，在“OpenWrt 源代码目录/mypackages/examples/mytest/src”目录下新建 shell 脚本 mytest，其内容如下：

```
1 #!/bin/sh /etc/rc.common
2
3 START=100
4 STOP=15
5
6 start() {
7     echo "Start mytest!"
8     service_start /usr/bin/mytest &
9 }
10
11 stop() {
12     echo "Stop mytest!"
13     service_stop /usr/bin/mytest
14 }
15
16 restart() {
17     echo "Restart mytest!"
18     service_stop /usr/bin/mytest
19     service_start /usr/bin/mytest &
20 }
```

第 1 行是特殊注释，表示使用“/etc/rc.common”脚本来提供一些基本函数，包含主函数及默认功能以及检查脚本执行等。

第 3 行指定启动顺序号，所有脚本按从小到大顺序启动；

第 4 行指定关闭顺序号，所有脚本按从小到大顺序关闭；

第 6 行定义了 start 函数用于启动软件；

第 7 行在终端显示信息；

第 8 行启动 mytest，&表示在后台运行；

第 11 行定义了 stop 函数用于关闭软件；

第 12 行在终端显示信息；

第 13 行关闭 mytest。

第 16-20 行定义了 restart 函数用于重启应用，实际为 stop 和 start 的结合。

为了能在安装软件包时能自动地将启动脚本复制到相应目录，赋予可执行权限并建立软链接，需要在对 Makefile 文件 Package/mytest/install 段进行修改。

```
define Package/mytest/install
    $(INSTALL_DIR) $(1)/etc/config
    $(INSTALL_CONF) ./src/mytest.config $(1)/etc/config/mytest
    $(INSTALL_DIR) $(1)/usr/bin
    $(INSTALL_BIN) $(PKG_BUILD_DIR)/mytest $(1)/usr/bin
    $(INSTALL_DIR) $(1)/usr/lib/lua/luci/controller
    $(INSTALL_CONF) ./src/other.lua $(1)/usr/lib/lua/luci/controller/other.lua
    $(INSTALL_DIR) $(1)/usr/lib/lua/luci/model/cbi
    $(INSTALL_CONF) ./src/mytest.lua $(1)/usr/lib/lua/luci/model/cbi/mytest.lua
    $(INSTALL_DIR) $(1)/etc/init.d
    $(INSTALL_BIN) ./src/mytest $(1)/etc/init.d/mytest
endef
```

重新编译并安装软件包后，在开发板的/etc/init.d 目录下可以看到 mytest 启动脚本：

```
root@jotale:/etc/init.d# ls
boot      done      led        network    sysctl    system     umount
cron      dropbear log         odhcpd     sysfixtime telnet
dnsmasq   firewall mytest     rpcd       sysntpd   uhttpd
```

在开发板的/etc/rc.d 目录下也可以看到 S100mytest 和 K15mytest 两个软链接已经自动建立。

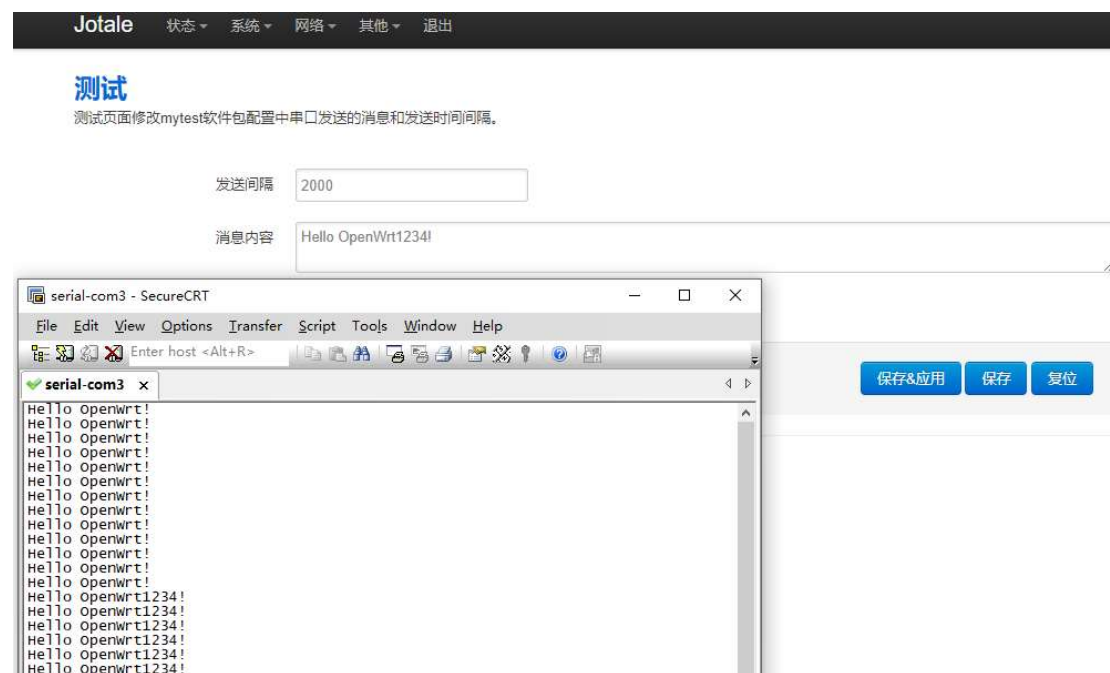
```
root@jotale:/etc/rc.d# ls
K15mytest    K98boot    S10system  S20network  S50uhttpd
K50dropbear K99umount  S11sysctl  S35odhcpd   S60dnsmasq
K85odhcpd   S00sysfixtime S12log     S50cron     S95done
K89log      S100mytest S12rpcd    S50dropbear S96led
K90network  S10boot    S19firewall S50telnet   S98sysntpd
```

登录开发板 web 配置页面将发送间隔改为 1000，断电重启，系统启动后即可看到 mytest 自动运行通过 RS232 串口输出信息。此外我们也可以在调试终端用“/etc/init.d/mytest start”、“/etc/init.d/mytest stop”、“/etc/init.d/mytest restart” 命令启动、关闭和重启应用。

至此，我们已经实现了软件包的开机自启动，但是由于软件包的配置是程序初始化时读入的，在程序进入 while 循环后通过 LuCI 改变的配置不会更新到程序，因此不会马上生效。为了使配置的改变能马上生效，我们需要在 mytest.lua 中重写 on_after_commit 函数：

```
local fs = require "nixio.fs"
m = Map("mytest", translate("测试"), translate("测试页面修改mytest软件包配置中串口发送的消息和发送时间间隔。"))
s = m:section(TypedSection, "mytest", "")
s.addremove = false
s.anonymous = true
view_period = s:option(Value, "period", translate("发送间隔"))
view_period.default = 0
view_output = s:option(TextView, "output", "消息内容")
view_output.rmemory = false
view_output.rows = 2
m.on_after_commit = function(self)
    luci.sys.call("/etc/init.d/mytest restart > /dev/null")
end
return m
```

重新编译软件包并在开发板上进行安装，安装过程中会提示 “ln: /etc/rc.d/S100mytest: File exists”，不用理会。登录开发板 web 配置界面，在“其他”->“测试”页面中修改发送间隔和消息内容，在串口终端可以观察到相应的变化。



5. 智能路由器应用实例

5.1 智能路由器媒体中心

围绕 OpenWrt 搭建一个家庭媒体中心，要求实现：

- U 盘（或用 TF 卡+读卡器模拟 U 盘）扩展存储容量并实现开机自动加载
- 通过 ftp 实现媒体文件的上传和下载
- BT 种子和磁力链接的远程下载（路由器直接下载到存储器）
- 远程音乐播放，将路由器作为音乐播放器，通过手机 app 控制音乐的播放
- DLNA 数字媒体服务器，将路由器作为 DLNA 的 DMS（Digital Media Server）服务器端，用来存储数据，为 DLNA 的 DMP 播放器提供数据。

相关参考：

存储扩展：

- 1、U 盘（或用 TF 卡+读卡器模拟 U 盘）格式化成 ext4

MiniTool Partition Wizard v12.3

- 2、LuCI 界面设置 U 盘自动加载（用 LuCI 界面）

<https://www.it610.com/article/1279936573621682176.htm>

远程下载：

- 1、ftp 服务器：vsftpd，已编译进内核

《JS76x8 开发板 OpenWrt 入门教程》，章节 3.4.1.3

百度以 openwrt 和 vsftpd 为关键词搜索

- 2、远程下载：aria2 + yaaw + luci-app-aria2，已编译进内核

<https://www.right.com.cn/forum/thread-135833-1-1.html>，此时你会发现 ftp 和 http 可以下载，但 bt 和磁力链接没法下载，解决的办法是增加 bt-tracker

<https://www.douban.com/note/654677252/>，自动更新 bt-tracker，你会发现脚本运行出错，因为缺少 ssl 支持，需要运行命令“opkg update”、“opkg install ca-certificates”和“opkg install wget”才能正常运行脚本。然并软，在脚本运行成功后你还是发现 bt 和磁力链接没有速度，仔细读下自动更新代码你会发现，代码会找到/etc/config/aria2 并在 option bt_tracker 后增加相应的地址，但/etc/config/aria2 并没有 option bt_tracker，因

此地址没有写入。解决办法是在/etc/config/aria2 最后面增加 option bt_tracker "并保存后再运行脚本，你会发现后面的地址更新了。

3、远程音乐播放

Openwrt 端 mpd-mini，安卓端 MPDroid（apk 安装文件 ftp 可下载），注意我们的声卡是 ALSA 的，因此/etc/mpd.conf 中 audio_output 里的 type 要设为 alsa，其余的设置可百度。

4、DLNA 数字媒体服务器

Openwrt 端：minidlna+luCI-app-minidlna+luCI-i18n-minidlna-zh-cn

客户端：Windows 10

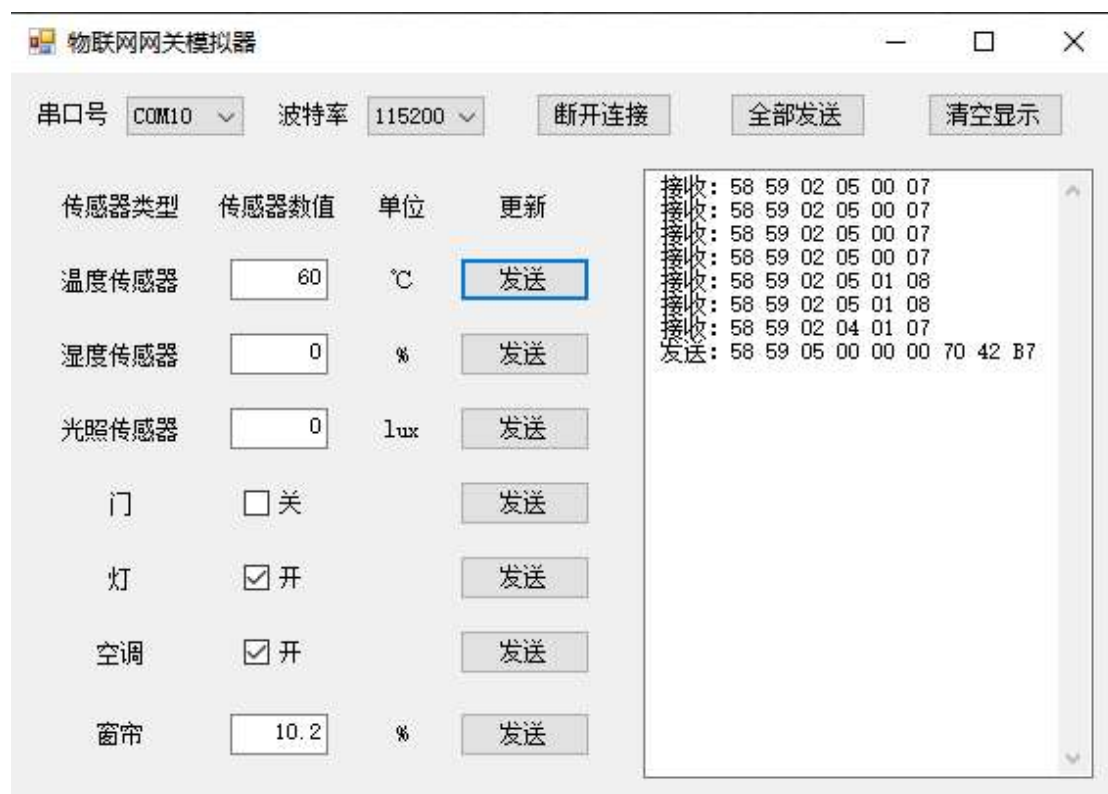
具体资料可百度搜索

5.2 智能物联网设备

在物联网设备应用中，OpenWrt 智能路由器一方面通过串口与模拟传感器连接，接收传感器信息并向设备发送控制命令；另一方面通过基于 TCP/IP 的 MQTT 协议与腾讯云的物联网平台连接，将传感器信息上传到云端，并从云端接收控制命令控制相应设备。

5.2.1 传感器模拟器

我们基于 C#开发了如下软件模拟温度、湿度、光照三种传感器和门、灯、空调、窗帘 4 种执行器。该软件通过 USB 转串口线连接到 OpenWrt 开发板，选择正确串口号（查看设备管理器）和波特率（115200bps），点击开始“开始连接”按钮即可连接开发板（连接后按钮名称会变为“断开连接”）。点击每个设备右边的“发送”按钮，即可将相应的信息通过串口发送到 OpenWrt 的串口，点击“全部发送”按钮则将所有设备信息通过串口发送到 OpenWrt 开发板的串口。



串口通讯采用“同步码（2 字节） + 长度（1 字节） + 设备标识（1 字节） + 数据（1-4 字节） + 校验码（1 字节）”的简单协议，其中：

- 2 字节同步码固定为 0x58, 0x59，在初始情况下连续接收到这两字节则视为一帧数据的

开始。

- 1 字节的长度标识设备标识+数据的字节数。
- 1 字节设备标识用于识别设备，0-6 分别表示温度传感器、湿度传感器、光照传感器、门、灯、空调、窗帘这 7 种设备。
- 1-4 字节的数据表示每个设备的数据值，其中温度传感器、湿度传感器、光照传感器和窗帘的数值为 4 个字节的 float 数据，门、灯、空调为 1 个字节的布尔量（0 和 1 分别表示关和开）。
- 1 字节校验和为累加和校验，其值为长度、设备标识、数据这 3 部分字节数据的累加和。
- 通过“发送”和“全部发送”按钮软件可以把这 7 种设备的信息发送到 OpenWrt 开发板模拟数据采集功能，而开发板也可以用相同的数据格式发送设备控制命令到软件模拟设备控制功能。

5.2.2 智能路由器与物联网设备模拟器的通讯

智能路由器通过标准 Linux 串口（/dev/ttyS1）与 PC 机上运行的物联网设备模拟软件通讯，串口初始化设置与 4.3.4 中的串口设置一致，串口的读写可参照《嵌入式 Linux 应用程序开发详解-第 6 章 文件 IO 编程》。

串口的接收采用 read 函数实现，Linux 在操作系统层面将串口数据进行了缓冲，我们在程序中调用 read 函数读取串口时，如果缓冲区中的数据多于要读取的数据，则只读取参数指定的数据量，如果缓冲区中的数据少于要读取的数据，则读取全部数据。read 函数的返回值为实际读取的字节数。下面的代码调用 read 函数将串口数据读取到 buf 数组中，cnt 为实际读取的字节数。读取到数据后我们即可对每一字节数据进行解析，首先识别两个字节的同步码，然后根据长度接收数据并计算累加校验和，最后将计算的校验码和接收到的校验码进行对比。如果两个校验码一致，则表明正确接收到了一帧数据，此时根据设备标识即可识别是哪个设备的数据并解析数据，同时将相应设备的 state 置为 eCHANGED 以通知相应函数。

```
void HandleDataFromComport(int fd)
{
    #define MAX_DATA_LENGTH 5
    unsigned int cnt, i;
    unsigned char buf[4096];
    static unsigned char recvData[MAX_DATA_LENGTH], *pData, checkSum, recvFlag = 0, recvLength;

    cnt = read(fd, buf, 4096);

    if (cnt == 0)
        return;

    for (i = 0; i < cnt; i++)
    {
        /*此处添加自己的代码解析串口接收的数据，接收到设备数据后讲传感器属性的state字段赋值为eCHANGED，通知相应函数将数据上传到云端*/
    }
}
```


串口信息的发送也与 4.3.4 类似，均是调用 write 函数实现，唯一的区别是按要 5.2.1 要求的帧格式写入数据。

5.2.3 腾讯云物联网设备端 C SDK 移植

腾讯物联网开发平台提供了设备端 C SDK (<https://github.com/tencentyun/qcloud-iot-explorer-sdk-embedded-c/releases>，也可到 ftp 下载 qcloud-iot-explorer-sdk-embedded-c-master.zip)，为物联网领域开发人员提供设备端快速接入云端，并和云端进行双向通信的能力。C SDK 采用模块化设计，分离核心协议服务与硬件抽象层，并提供灵活的配置选项和多种编译方式，适用于不同设备的开发平台和使用环境。对于具备网络通讯能力并使用标准 Linux/Windows 系统的设备。例如 PC/服务器/网关设备，及较高级的嵌入式设备，例如树莓派等，可直接在该设备上编译运行 SDK。

由于 OpenWrt 开发板使用的是 MIPS 指令的 CPU，因此需要修改编译配置进行交叉编译后才能在 OpenWrt 开发板上使用。具体交叉编译步骤如下：

- 下载 qcloud-iot-explorer-sdk-embedded-c-master.zip，解压并进入 SDK 根目录。
- 修改 make.settings，将标准的 gcc 编译器替换成 3.2.6 中安装的交叉编译器，具体修改如下：

```
# 编译类型: release/debug
BUILD_TYPE                = release
# Makefile编译打印 echo
DEBUG_MAKEFILE            = n

# 编译工具链
PLATFORM_CC                = mipsel-openwrt-linux-gcc
PLATFORM_AR                = mipsel-openwrt-linux-gcc-ar
PLATFORM_OS                = linux
PLATFORM_SSL               = mbedtls
PLATFORM_AT_DEV            = esp8266
```

- 在 SDK 根目录下输入 make 命令启动交叉编译，交叉编译完毕后在“SDK 根目录/output/release/lib”下的 5 个静态库文件即为使用 SDK 所需的文件，“SDK 根目录/output/release/include”和“SDK 根目录/sdk_src/internal_inc”为相应的头文件目录，为了使用这些库文件，我们需要把这 3 个文件夹均拷贝到我们自己要开发软件包的源代码根目录，并修改软件包的 Makefile 把库文件包含到软件包里。在编译时指定头文件目录，在链接时指定相应的静态库文件。


```

IOT_INC_DIR = -I$(PKG_BUILD_DIR)/include -I$(PKG_BUILD_DIR)/include/exports -I$(PKG_BUILD_DIR)/internal_inc
TARGET_CFLAGS += -Wall -Wno-error=sign-compare -Wno-error=format -pthread -DFORCE_SSL_VERIFY
IOT_LDFLAGS = -Wl,--start-group $(PKG_BUILD_DIR)/lib/libiot_sdk.a $(PKG_BUILD_DIR)/lib/libmbedtls.a $(PKG_BUILD_DIR)/lib/libmbedx509.a $(PKG_BUILD_DIR)/lib/libmbedcrypto.a $(PKG_BUILD_DIR)/lib/libiot_platform.a -Wl,--end-group

define Build/Compile
    $(TARGET_CC) $(TARGET_CFLAGS) $(IOT_INC_DIR) -o $(PKG_BUILD_DIR)/gateway.o -c $(PKG_BUILD_DIR)/gateway.c
#    $(TARGET_CC) $(TARGET_CFLAGS) -o $(PKG_BUILD_DIR)/config.o -c $(PKG_BUILD_DIR)/config.c
    $(TARGET_CC) $(TARGET_CFLAGS) -o $(PKG_BUILD_DIR)/comport.o -c $(PKG_BUILD_DIR)/comport.c
    $(TARGET_CC) $(TARGET_LDFLAGS) -o $(PKG_BUILD_DIR)/$1 $(PKG_BUILD_DIR)/gateway.o -luci $(PKG_BUILD_DIR)/comport.o
$(IOT_LDFLAGS)
endef

```

- “/output/release/bin” 目录提供了 samples 下面各个例程的可执行文件，大家可对照例程理解 SDK 的使用。

5.2.4 云端产品建立

云端产品的建立可参照物联网开发平台→快速入门→智能灯接入指引里的步骤 (<https://cloud.tencent.com/document/product/1081/41155>)，其具体步骤如下：

- 登录物联网开发平台控制台 (<https://console.cloud.tencent.com/iotexplorer>)，选择“新建项目”，在新建项目页面填写项目基本信息（内容自定），点击“保存”按钮完成新建项目。
- 进入该项目的产品列表页面，单击“新建产品”按钮，填写产品基本信息并保存，即可完成产品新建。

新建产品

×

产品名称 *

智能路由器

支持中文、英文、数字、下划线的组合，最多不超过20个字符

产品品类

用户自定义

设备类型

设备

网关

子设备

认证方式

密钥认证

证书认证

通信方式

Wi-Fi

2G/3G/4G

5G

BLE

LoRaWAN

其它

①

数据协议

数据模板

自定义透传

- 产品新建成功后，点击产品即可进入“数据模板”界面，由于我们选择了用户自定义，所有数据模板需要自己手动建立，如果在产品品类里选择标准设备，系统会自动生成标准设备对应的数据模板。按照 5.2.1 中模拟的设备，我们可以在自定义功能页面建立对

应的属性，其中的标识符用来识别各个属性。

自定义功能

新建功能 您可以通过自定义功能按需定义功能						
功能类型	功能名称	标识符	数据类型	读写类型	数据定义	操作
属性	湿度	humidity	浮点型	只读	数值范围: 0-100 初始值: 0 步长: 0.1 单位:	编辑 删除
属性	温度	temperature	浮点型	只读	数值范围: -40-155 初始值: 25 步长: 0.1 单位:	编辑 删除
属性	门	door	布尔型	只读	0 - 关 1 - 开	编辑 删除
属性	光照	illumination	浮点型	只读	数值范围: 0-10000 初始值: 0 步长: 0.1 单位:	编辑 删除
属性	灯	light	布尔型	读写	0 - 关 1 - 开	编辑 删除
属性	空调	ac	布尔型	读写	0 - 关 1 - 开	编辑 删除
属性	窗帘	curtain	浮点型	读写	数值范围: 0-100 初始值: 0 步长: 0.1 单位:	编辑 删除

- 在设备调试界面点击“新建设备”输入设备名称即可完成设备新建。

数据模板 > 设备开发 > 交互开发 > 4 设备调试 > 5 批量投产

设备调试提供真实、虚拟设备调试功能，便于测试设备上报、接收数据是否正常，可创建测试设备进行调试，开发中产品最多创建50个设备

新建设备 虚拟设备调试

设备名称

设备名称

Q

设备名称	状态	激活时间	最后上线时间	操作
router1	在线	2021-03-17 08:47:54	2021-03-18 21:58:11	调试 二维码 子设备管理 删除

- 为了测试设备并更好地理解 MQTT 通讯，我们可以使用 MQTT.fx 模拟设备与云端通讯，具体调试步骤可以参照 <https://cloud.tencent.com/document/product/1081/46507>，MQTT.fx 也可以到 ftp 进行下载。

5.2.5 连接 MQTT 云端服务并构造数据模板客户端

设备与 IoT 云端的连接通过 SDK 提供的 IOT_Template_Construct(TemplateInitParams *pParams, void *pMqttClient)实现，该函数构造数据模板客户端 Data_template_client 并连接 MQTT 云端服务，其中：pParams 为连接需要的参数；pMqttClient 为已经建立的 MQTT 连接句柄，正常情况下 MQTT 连接与数据模板客户端是一起建立的，因此改参数输入为 NULL，函数先创建 MQTT 连接后建立数据模板客户端。在 main 函数的初始化代码中，连接 MQTT 云端服务并构造数据模板客户端的具体代码实现如下：

```
//配置连接参数，连接MQTT云端服务并构造数据模板客户端
ConfigInitParams(&initParams);
void *client = IOT_Template_Construct(&initParams, NULL);
if (client != NULL) {
    Log_i("Cloud Device Construct Success");
} else {
    Log_e("Cloud Device Construct Failed");
    return QCLOUD_ERR_FAILURE;
}
```

其中：initParams 为包含连接参数的结构体，其参数由 ConfigInitParams 函数进行初始化：

```
#define REGION "china"
#define DEVICE_NAME "router1"
#define PRODUCT_ID "JM5OCF82QZ"
#define DEVICE_SECRET "6SYQA4SJvHXpWbMe96KXqw=="

static void ConfigInitParams(TemplateInitParams *initParams)
{
    initParams->region = REGION;
    initParams->device_name = DEVICE_NAME;
    initParams->product_id = PRODUCT_ID;
    initParams->device_secret = DEVICE_SECRET;
    initParams->command_timeout = QCLOUD_IOT_MQTT_COMMAND_TIMEOUT;
    initParams->keep_alive_interval_ms = QCLOUD_IOT_MQTT_KEEP_ALIVE_INTERVAL;
    initParams->auto_connect_enable = 1;
    initParams->event_handle.h_fp = event_handler;
    initParams->usr_control_handle = NULL;
}
```

其中：REGION 目前固定为“china”，DEVICE_NAME、PRODUCT_ID、DEVICE_SECRET 与我们建立的设备信息中的设备名称、产品 ID、设备密钥对应，需要改成你们自己设备的参数。在例程中这些参数我们通过宏定义给定，最终这 4 个参数需要通过 LuCI 界面进行配置。

设备信息	设备属性	设备日志	设备日志(透传数据)	设备事件	设备行为	设备上下线日志	在线调试	扩展
设备信息								
设备名称	router1	所属产品	router	设备创建时间	2021-03-17 08:44:27			
设备密钥	6SYQA4SJvHXpWbMe96KXqw==	产品ID	JM5OCF82QZ	设备状态	离线			
激活时间	2021-03-17 08:47:54	最后上线时间	2021-03-18 21:58:11	固件版本	-			

5.2.6 注册设备数据模板属性

注册数据模板属性的目的是让 SDK 将云端定义的数据模板跟设备端的数据关联起来，设备端 SDK 提供的用于定义数据模板属性的数据结构如下：

<pre>typedef struct { DeviceProperty data_property; eDataState state; } sDataPoint;</pre>	<pre>typedef struct _JSONNode { char *key; // 对应云端属性的标识符 void *data; // 属性的数值 union { uint16_t data_buff_len; //数据长度 uint16_t struct_obj_num; //结构体成员数 }; JsonDataType type; // 数据类型 } DeviceProperty;</pre>
--	--

其中：key 和 data 为指向标识符和数值的指针；type 表明该属性的数据类型，有 TYPE_TEMPLATE_INT、TYPE_TEMPLATE_FLOAT、TYPE_TEMPLATE_BOOL、TYPE_TEMPLATE_STRING、TYPE_TEMPLATE_TIME、TYPE_TEMPLATE_JOBJECT 等选项；state 为属性状态，eNOCHANGE 表示属性值没有改变，eCHANGED 在 SDK 中表示属性值发生了变化（无论是云端改变的还是设备端改变的），这容易发生混淆，因此在我们的程序中，我们用 eCHANGED 表示因设备端引发的属性值改变，用 cCHANGED 表示因云端引发的属性值改变。在云端我们总共定义了 7 个属性，对应的设备端我们也要定义 7 个属性数据，并定义保存这 7 个数值的结构体：

```
#define TOTAL_PROPERTY_COUNT 7
static sDataPoint dataTemplates[TOTAL_PROPERTY_COUNT];

typedef struct _ProductDataDefine {
    TYPE_DEF_TEMPLATE_FLOAT    m_temperature;
    TYPE_DEF_TEMPLATE_FLOAT    m_humidity;
    TYPE_DEF_TEMPLATE_FLOAT    m_illumination;
    TYPE_DEF_TEMPLATE_BOOL     m_door;
    TYPE_DEF_TEMPLATE_BOOL     m_light;
    TYPE_DEF_TEMPLATE_BOOL     m_ac;
    TYPE_DEF_TEMPLATE_FLOAT    m_curtain;
} ProductDataDefine;
static ProductDataDefine productData;
```

上述两个数据在 InitDataTemplate 函数中进行初始化：


```

static void InitDataTemplate(void)
{
    memset((void *)&productData, 0, sizeof(productData));

    dataTemplates[0].data_property.key = "temperature";
    dataTemplates[0].data_property.data = &productData.m_temperature;
    dataTemplates[0].data_property.type = TYPE_TEMPLATE_FLOAT;
    dataTemplates[0].state = eNOCHANGE;

    dataTemplates[1].data_property.key = "humidity";
    dataTemplates[1].data_property.data = &productData.m_humidity;
    dataTemplates[1].data_property.type = TYPE_TEMPLATE_FLOAT;
    dataTemplates[1].state = eNOCHANGE;

    dataTemplates[2].data_property.key = "illumination";
    dataTemplates[2].data_property.data = &productData.m_illumination;
    dataTemplates[2].data_property.type = TYPE_TEMPLATE_FLOAT;
    dataTemplates[2].state = eNOCHANGE;

    dataTemplates[3].data_property.key = "door";
    dataTemplates[3].data_property.data = &productData.m_door;
    dataTemplates[3].data_property.type = TYPE_TEMPLATE_BOOL;
    dataTemplates[3].state = eNOCHANGE;

    dataTemplates[4].data_property.key = "light";
    dataTemplates[4].data_property.data = &productData.m_light;
    dataTemplates[4].data_property.type = TYPE_TEMPLATE_BOOL;
    dataTemplates[4].state = eNOCHANGE;

    dataTemplates[5].data_property.key = "ac";
    dataTemplates[5].data_property.data = &productData.m_ac;
    dataTemplates[5].data_property.type = TYPE_TEMPLATE_BOOL;
    dataTemplates[5].state = eNOCHANGE;

    dataTemplates[6].data_property.key = "curtain";
    dataTemplates[6].data_property.data = &productData.m_curtain;
    dataTemplates[6].data_property.type = TYPE_TEMPLATE_FLOAT;
    dataTemplates[6].state = eNOCHANGE;
}

```

上述初始化函数中，数组每个元素的 key 均与云端各属性的标识符对应，而 data 指向数值结构体中对应数值的地址，**注意此时 productData 中所有的数值均为零。**

设备数据模板属性的注册通过 IOT_Template_Register_Property 函数实现，我们在 ConfigDataTemplate 函数中调用该函数对每一个数据模板属性进行注册。

```

static int ConfigDataTemplate(void *client)
{
    int i, rc;
    for(i = 0; i < TOTAL_PROPERTY_COUNT; i++)
    {
        rc = IOT_Template_Register_Property(client, &dataTemplates[i].data_property, OnControlMsgCallback);
        if (rc != QCLOUD_RET_SUCCESS) {
            rc = IOT_Template_Destroy(client);
            Log_e("register device data template property failed, err: %d", rc);
            return rc;
        } else {
            Log_i("data template property=%s registered.", dataTemplates[i].data_property.key);
        }
    }
    return QCLOUD_RET_SUCCESS;
}

```

最后在 main 函数的初始化阶段，我们调用这两个函数实现设备数据模板属性的注册。

```

//注册设备数据模板属性
InitDataTemplate();//初始化属性数据
rc = ConfigDataTemplate(client);
if (rc == QCLOUD_RET_SUCCESS) {
    Log_i("Register data template property Success");
} else {
    Log_e("Register data template property Failed: %d", rc);
    goto exit;
}

```

5.2.7 获取云端数据模板属性

在 5.2.6 中我们提到, `productData` 中所有的数值均被初始化为零, 在初始化时需要获取各属性的数值, 获取属性数值有两种方法:

- 读取各传感器和执行器的属性数值, 并更新到云端;
- 从云端读取数据模板属性, 并更新到传感器和执行器。

这两种方式可以根据需要选择其中一种, 从云端读取数据模板属性主要通过 `IOT_Template_GetStatus` 函数实现, 具体代码如下:

```
//从云端读取数据模板属性, 可根据需要选择这种方式对属性数值进行初始化;
//也可直接读取传感器或执行器并更新到云端, 此时不需要下面的代码。
rc = IOT_Template_GetStatus(client, GetStatusReplyAckCb, &ackRequest, QCLOUD_IOT_MQTT_COMMAND_TIMEOUT);
if (rc != QCLOUD_RET_SUCCESS) {
    Log_e("Get data status fail, err: %d", rc);
}
while(ACK_NONE == ackRequest)
{
    IOT_Template_Yield(client, 200);
}
Log_i("%f, %f, %f", productData.m_temperature, productData.m_humidity, productData.m_illumination);
```

其中: `client` 为 5.2.5 中连接成功后返回的句柄; `GetStatusReplyAckCb` 为更新完成后的回调函数, `ackRequest` 为应答请求, `QCLOUD_IOT_MQTT_COMMAND_TIMEOUT` 为超时参数。调用 `IOT_Template_GetStatus` 后需要执行 `IOT_Template_Yield` 函数进行 MQTT 报文读取和消息处理; 当收到云端数据后该函数调用 `GetStatusReplyAckCb` 对接收的数据进行处理。接收到的数据以 json 格式在 `pReceivedJsonDocument` 中给出, 在函数中调用 `LITE_json_value_of` 函数逐级对 json 数据进行处理, 以提取各属性的值。此处我们只解析了温度的属性值, 其他的属性值解析方法与温度类似。

```
static void GetStatusReplyAckCb(void *pClient, Method method, ReplyAck replyAck, const char *pReceivedJsonDocument, void *pUserData)
{
    Request *request = (Request *)pUserData;
    if (*((ReplyAck *)request->user_context) == ACK_ACCEPTED)
        IOT_Template_ClearControl(pClient, request->client_token, NULL, QCLOUD_IOT_MQTT_COMMAND_TIMEOUT);
    else
        *((ReplyAck *)request->user_context) = replyAck;
    printf("Received Json Document = %s", pReceivedJsonDocument);
    char *data = LITE_json_value_of("data", pReceivedJsonDocument);
    if (data != NULL)
    {
        char *reported = LITE_json_value_of("reported", data);
        if(reported != NULL)
        {
            if(reported != NULL)
            {
                char *tmpr = LITE_json_value_of("temperature", reported);
                if(tmpr != NULL)
                    productData.m_temperature = atof(tmpr);
            }
        }
    }
}
```

5.2.8 向云端上报数据

数据模板属性的上报通过 `IOT_Template_Report` 函数实现, 我们在 `ReportDataToCloud` 函数中首先逐个检查每个数据模板属性的状态是否被传感器或执行器改变, 然后通过

IOT_Template_JSON_ConstructReportArray 函数封装成 json 格式数据，最后将 json 数据通过 IOT_Template_Report 函数发送到云端，发送成功后触发 OnReportReplyCallback 回调函数进行相应处理。ReportDataToCloud 运行在 main 函数的 while 循环中，其他程序只需要改变属性的值并将属性对应的 state 变量赋值成 eCHANGED，即可将数据上报云端。

```
void ReportDataToCloud(void *pClient)
{
    int i, j = 0, rc;
    DeviceProperty *pReportDataList[TOTAL_PROPERTY_COUNT];
    //查找要上报的数据
    for(i = 0; i < TOTAL_PROPERTY_COUNT; i++)
    {
        if(dataTemplates[i].state == eCHANGED)
        {
            pReportDataList[j++] = &(dataTemplates[i].data_property);
            dataTemplates[i].state = eNOCHANGE;
        }
    }
    //没有数据上报, 返回
    if(j == 0)
        return;

    rc = IOT_Template_JSON_ConstructReportArray(pClient, dataReportBuffer, dataReportBufferSize, j, pReportDataList);
    if(rc == QCLOUD_RET_SUCCESS)
    {
        rc = IOT_Template_Report(pClient, dataReportBuffer, dataReportBufferSize, OnReportReplyCallback, NULL,
        QCLOUD_IOT_MQTT_COMMAND_TIMEOUT);
        if(rc == QCLOUD_RET_SUCCESS)
            Log_i("Data template report success");
        else
            Log_e("Data template report failed, err: %d", rc);
    }else{
        Log_e("Data template construct failed, err: %d", rc);
    }
}

void OnReportReplyCallback(void *pClient, Method method, ReplyAck replyAck, const char *pJsonDocument, void *pUserData)
{
    Log_i("Receive report reply(ack=%d): %s", replyAck, pJsonDocument);
}
```

5.2.9 响应云端下行的数据模板属性控制消息

在 5.2.6 注册设备数据模板属性时，我们将 OnControlMsgCallback 注册成收到云端控制消息后的回调函数。连接建立后，我们可以在物联网开发平台的设备在线调试界面发送属性的下发控制命令，设备端接收到命令将会触发 OnControlMsgCallback 函数进行处理。

在 OnControlMsgCallback 函数中，我们根据标识符找到相应的数据模板属性，并将其 state 变量赋值成 cCHANGED，通知其他函数该属性收到了下发控制指令。

```
static void OnControlMsgCallback(void *pClient, const char *pJsonValueBuffer, uint32_t valueLength,
                                DeviceProperty *pProperty)
{
    int i = 0;
    for (i = 0; i < TOTAL_PROPERTY_COUNT; i++)
    {
        if (strcmp(dataTemplates[i].data_property.key, pProperty->key) == 0)
        {
            dataTemplates[i].state = cCHANGED;
            Log_i("Property=%s changed: %d", pProperty->key, *((int *)dataTemplates[i].data_property.data));
            return;
        }
    }
    Log_e("Property=%s no match", pProperty->key);
}
```

在 main 函数的 while 循环中，HandleControlMsgFromCloud 函数逐个检查每个属性的 state 变量，并将被下发控制命令改变了的属性的值通过串口的 write 函数更新到 PC 端的传感器和执行器模拟软件；此外对于下发的控制指令，设备必须回复，否则设备每次上线，云端都会重发该控制指令。回复通过 IOT_Template_ControlReply 函数实现。

```
void HandleControlMsgFromCloud(int fd, void *pClient)
{
    int i, len, j, rc;
    unsigned char buf[10] = {0x58, 0x59}, recvd = 0;
    sReplyPara replyPara;

    for(i = 0; i < TOTAL_PROPERTY_COUNT; i++)
    {
        if(dataTemplates[i].state == cCHANGED)
        {
            recvd = 1;
            dataTemplates[i].state = eNOCHANGE;
            buf[3] = i;
            switch(i)sg_subscribe_event_result
            {
                case 0:
                case 1:
                case 2:
                case 6:
                    len = 4;
                    break;
                case 3:
                case 4:
                case 5:
                    len = 1;
                    break;
            }
            memcpy(buf+4, dataTemplates[i].data_property.data, len);
            buf[2] = len + 1;
            buf[buf[2]+3] = 0;
            for(j = 2; j < buf[2]+3; j++)
                buf[buf[2]+3] += buf[j];
            write(fd, buf, buf[2]+4);
        }
    }
    if(recvd)
    {
        printf("Replying control msg.");
        memset((char *)&replyPara, 0, sizeof(sReplyPara));
        replyPara.code = eDEAL_SUCCESS;
        replyPara.timeout_ms = QCLOUD_IOT_MQTT_COMMAND_TIMEOUT;
        replyPara.status_msg[0] = '\0';
        rc = IOT_Template_ControlReply(pClient, dataReportBuffer, dataReportBufferSize, &replyPara);
        if(rc==QCLOUD_RET_SUCCESS)
            Log_d("Control msg reply success.");
        else
            Log_e("Control msg reply failed, err: %d.", rc);
    }
}
```